Lab 3: Deploy

A key facet of healthy ALM is separating changes in development from those in testing and in production. Pipelines in Power Platform deploys the latest version of your solution across these environments, with the ability to add secure approvals and review changes before they're deployed. In this lab, you'll deploy from **Development 1** to **Test**, using the **Pipelines Host** you verified in Lab 1. Production stages may be added in a real-world pipeline.

Prerequisites:

- Complete Lab 2 (the Zava Quality Operations solution in **Development 1**, with the Quality Incident table, the Zava Quality Guide agent and its zava-quality-triage skill, and the Zava Quality Incident Tracker code app)
- If you did not complete Lab 2, download ZavaQualityOperations.zip from GitHub and import this unmanaged solution into **Development 1** (**Solutions** > **Import solution**). Turn on code apps in **Development 1** first (next item).
- Make sure Power Apps code apps are enabled in **Development 1** and **Test** before you import or deploy the solution: in the Power Platform admin center, open each environment > **Settings** > **Product** > **Features**, turn on **Power Apps code apps**, and click **Save**. If this setting is off, the code app is skipped during import and deployment without an error.

Lab 3 Tasks

- Types of Pipelines
- Create a Platform Host pipeline
    - Deploy using Platform Host pipeline
- Create a Custom Host pipeline
    - Verify the Pipelines Host environment
    - Configure environment records for the pipeline
    - Configure a basic pipeline
    - Deploy using a Custom Host pipeline
- Extend pipelines (optional, but highly recommended)
    - Gated extensions
    - Create an Application User
    - Create Sharing Security Group
    - Create application user in Dataverse
    - Create an approval + SPN delegated deployment flow
    - Update flow after solution import
    - Update pipeline to run as delegated
    - Run SPN delegated pipeline
- Redeploy a prior version

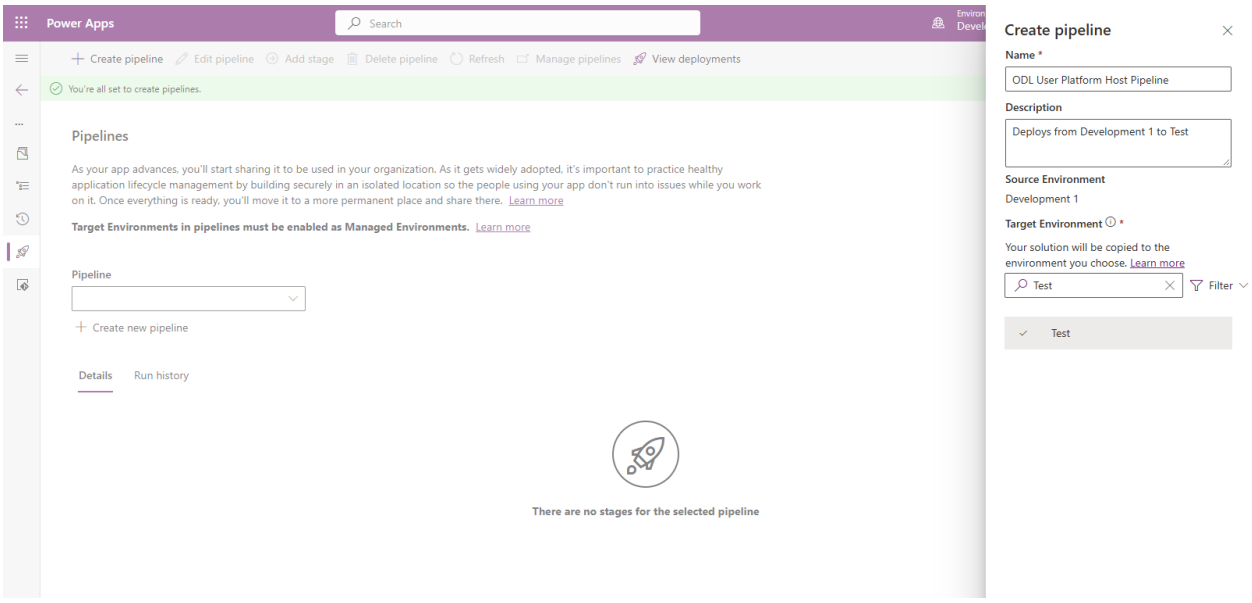
There are **two** ways to configure Power Pipelines—

Platform host- Out of the box tenant-wide pipelines ghosted under a “platform host” configuration. These pipelines are configured by makers for personal use. Personal pipelines cannot be extended, nor can they be shared with other makers in the tenant.

Custom host- Pipelines configuration hosted under a custom environment, where Power Pipelines modules are installed by admins. Custom pipelines are centrally governed by admins and shared with makers for use.

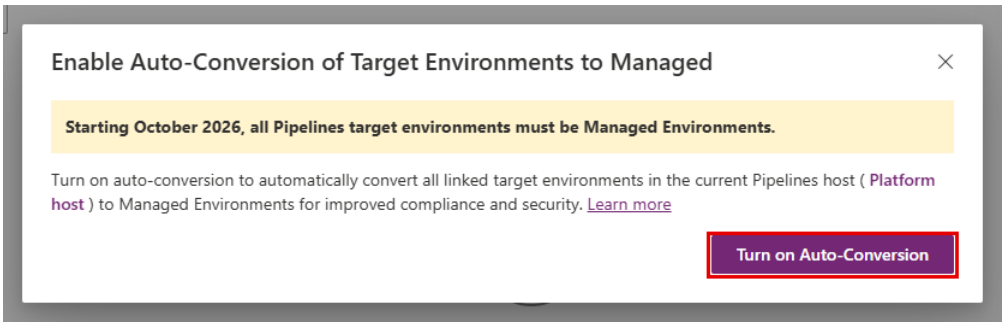
Create a Platform Host Personal Pipeline

1. Navigate to **Development 1**. This is where you built the solution you want to deploy.
2. Click on **Solutions** tab in the left-hand navigation blade
3. Once in the Solutions page, open the unmanaged **Zava Quality Operations** solution
4. On the left-hand navigation panel, click on the **rocket icon** for Power Pipelines page
5. On the Pipelines page, select **+ Create new pipeline**
6. In the Create pipeline pop out wizard, provide the following details:
    a. **Name:** <Your Name> Platform Host Pipeline
    b. **Description:** Include description of which environments this pipeline will deploy to
    c. **Target environment:** select **Test**
    d. Click **Save**



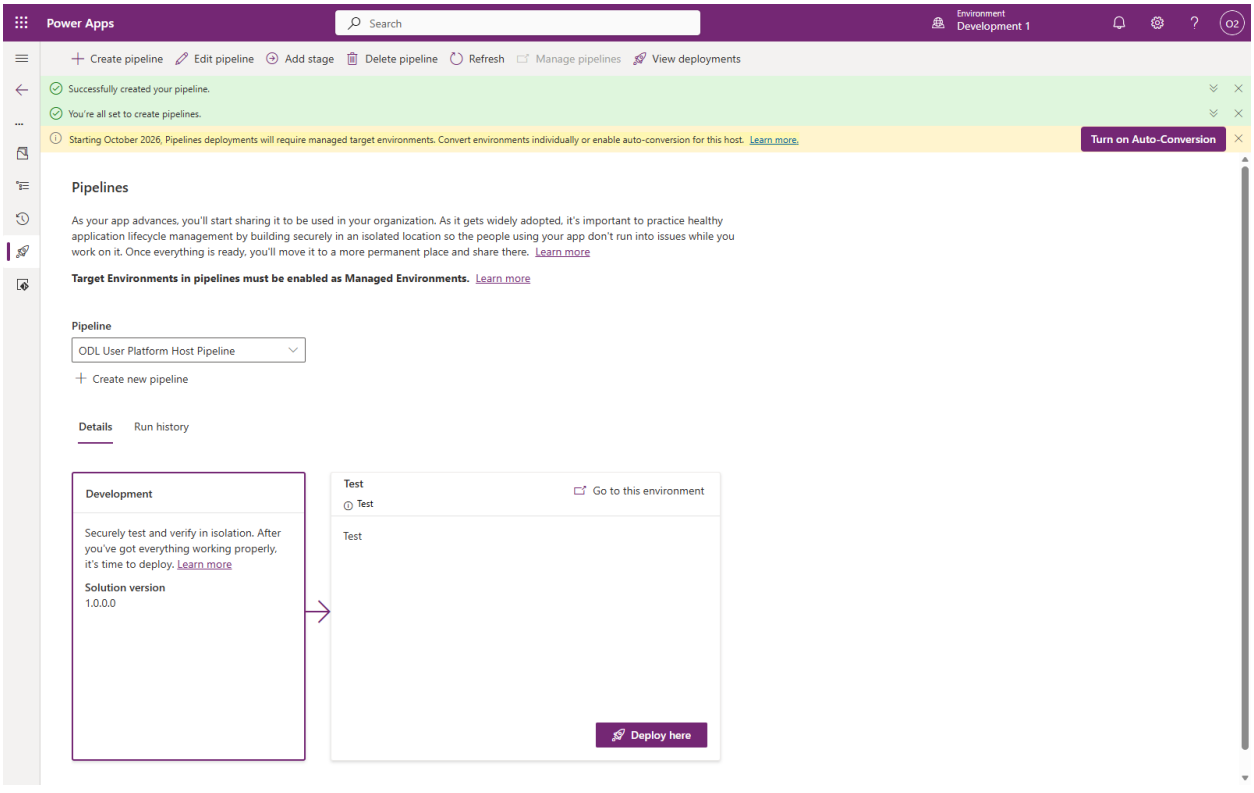
7. If **Enable Auto-Conversion of Target Environments to Managed** appears, click **Turn on Auto-Conversion**. The pipeline is saved once you do; closing the dialog doesn’t save it. Pipelines only deploy to Managed Environments, and auto-conversion lets the platform

host convert target environments for you. You still confirm the conversion of **Test** when you deploy.



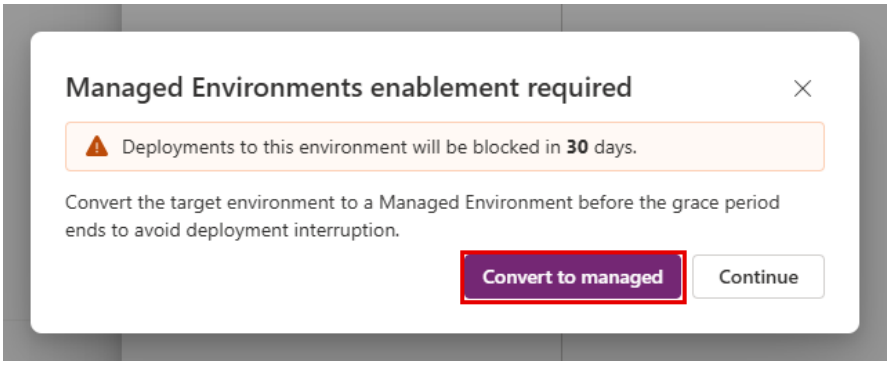
Deploy using a Platform Host pipeline

8. You're now ready to use this Pipeline to deploy your solutions! If your specific deployment process requires an additional environment, click on Add Stage button to add additional stages.

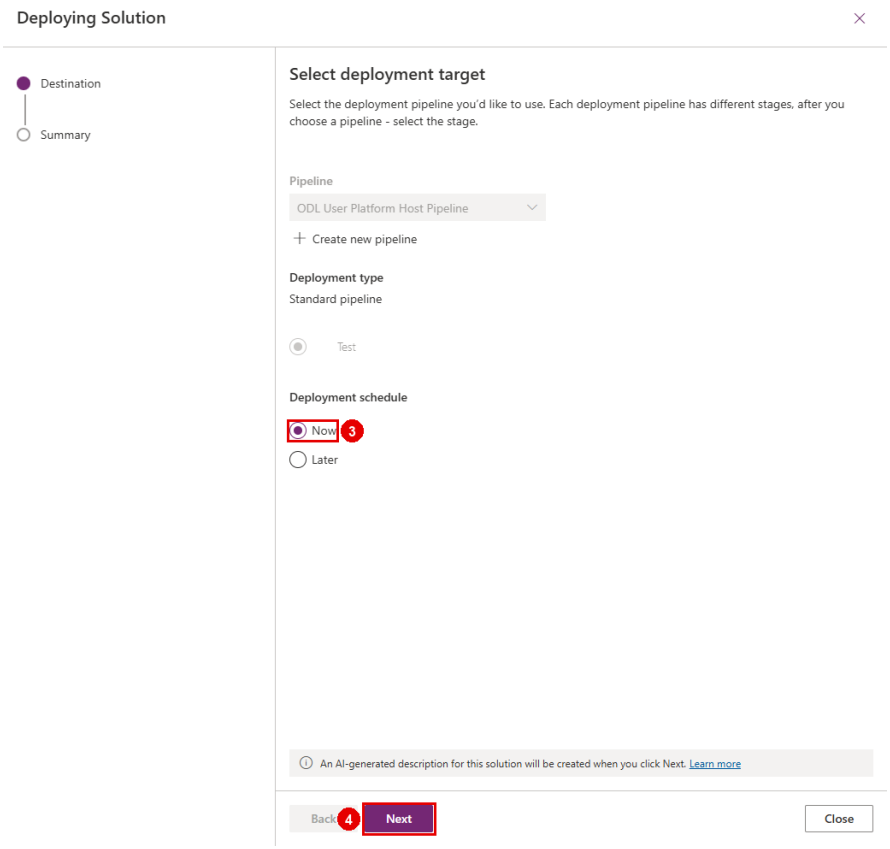


9. On the **Test** stage, click **Deploy here** to initiate the solution deployment to **Test**.

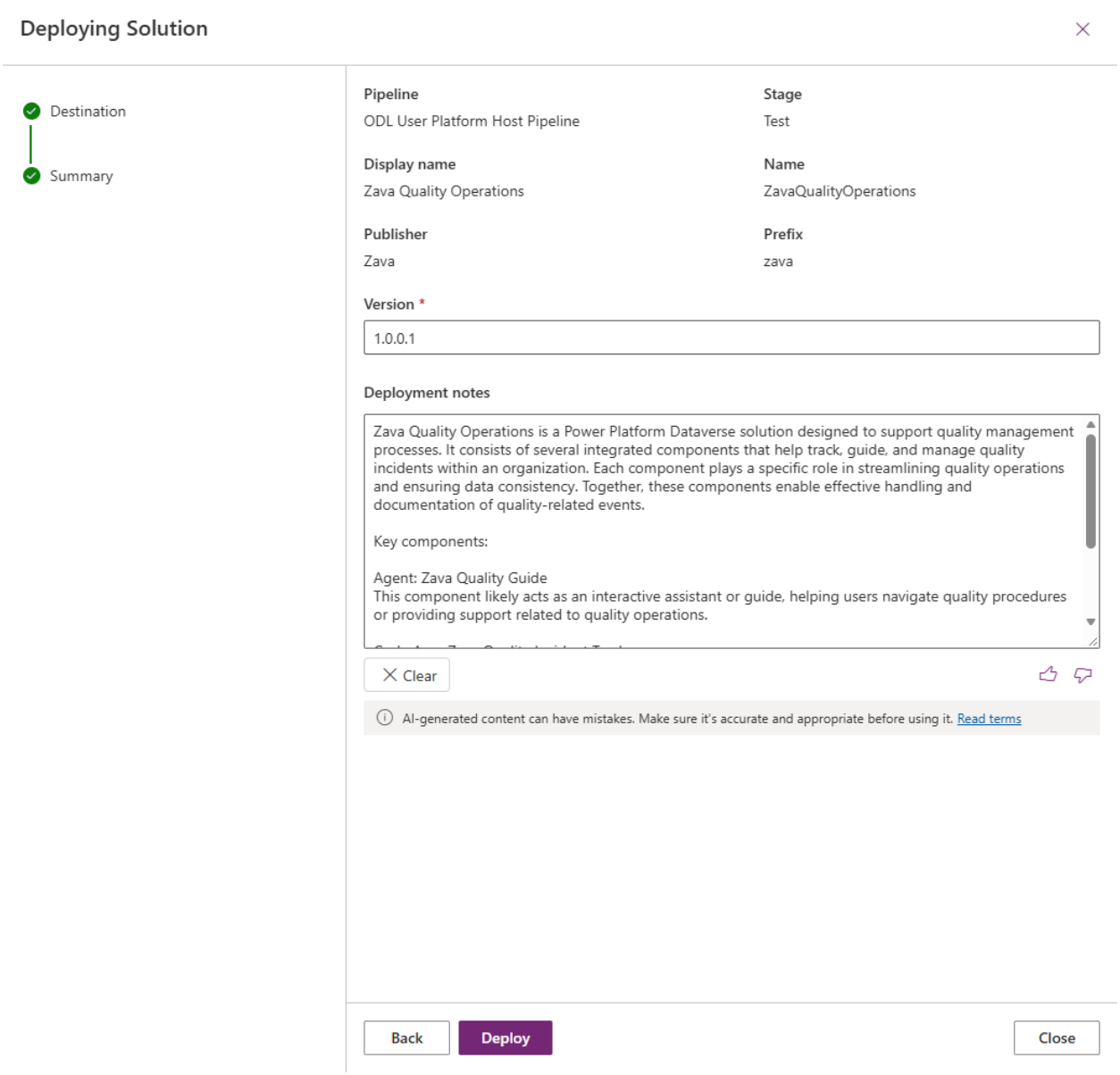
10. **Test** isn't a Managed Environment yet, so a **Managed Environments enablement required** popup appears. Click **Convert to managed**. Pipelines only deploy to Managed Environments.



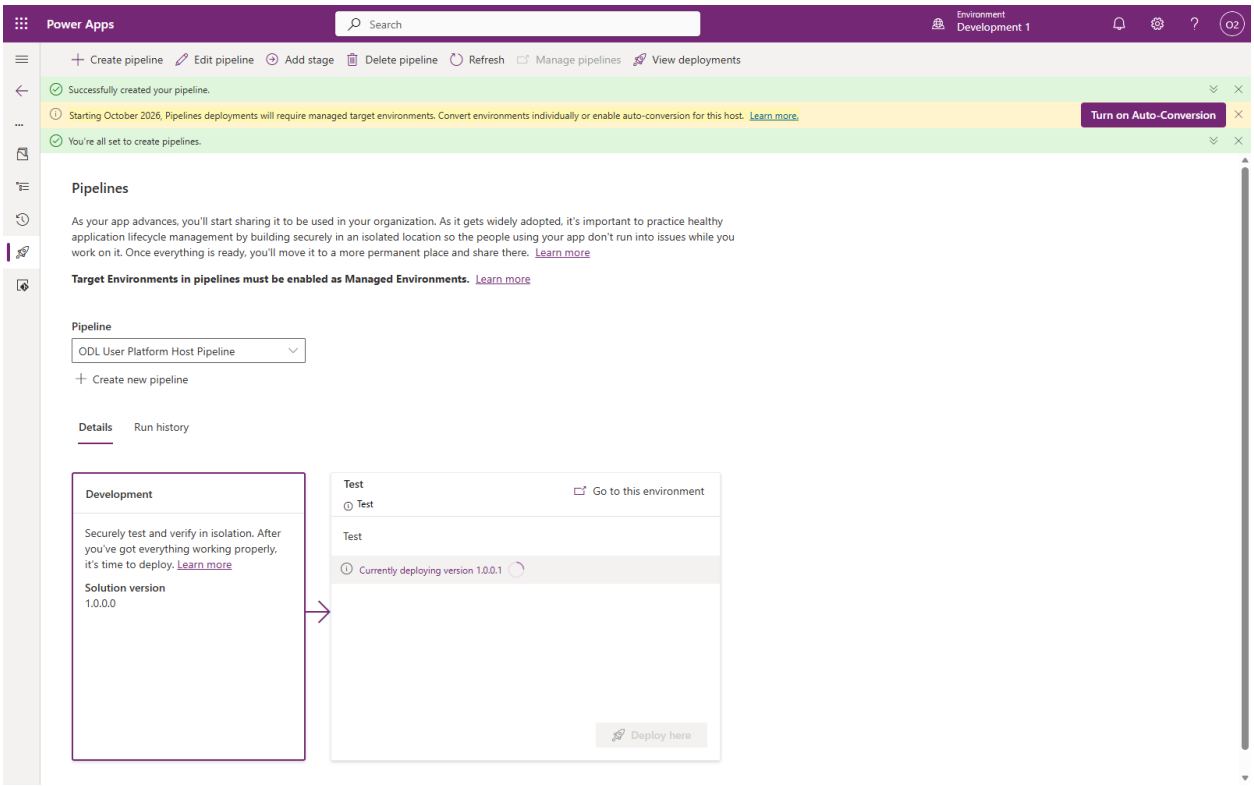
11. Under **Deployment schedule**, keep **Now**. You can schedule deployments by selecting **Later** and choosing a date and time when you'd like to automatically deploy.
12. Click **Next**



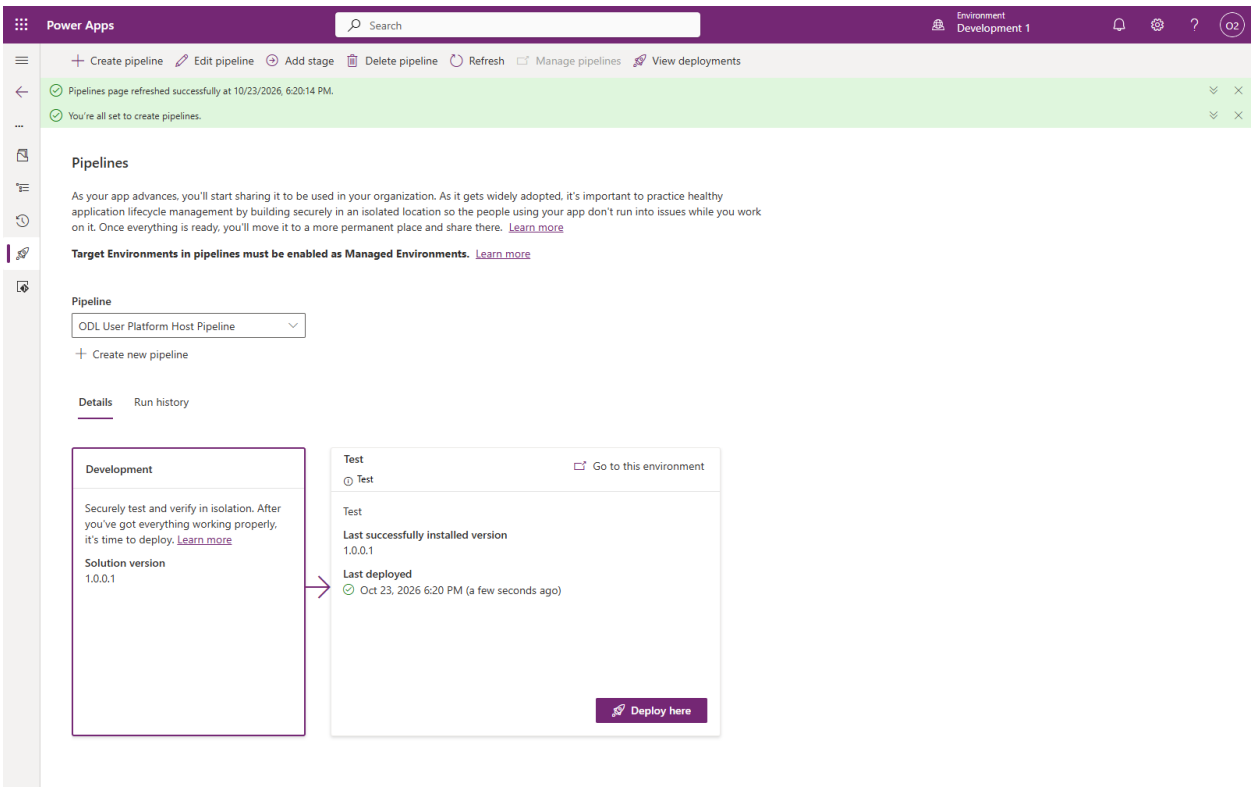
13. In the summary section, notice the **Deployment Notes** section was populated using AI-generated summary of your solution's capabilities.
14. Click **Deploy** to begin your deployment



Your **Test** stage will update with the progress of the deployment:



When the deployment completes, you can see the success message on the stage:



Remember: Platform Host pipelines are not shareable, nor can they be extended to support approvals or delegations. Continue to the Custom Host Pipelines section to create extendable and shareable pipelines.

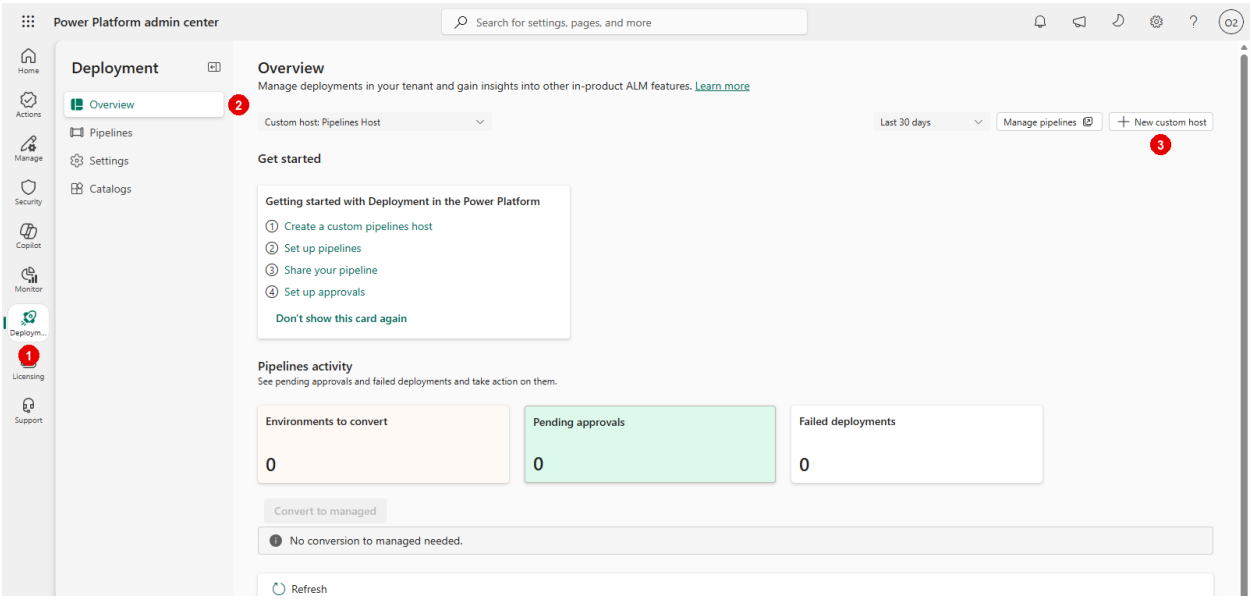
Create a Custom Host Pipeline

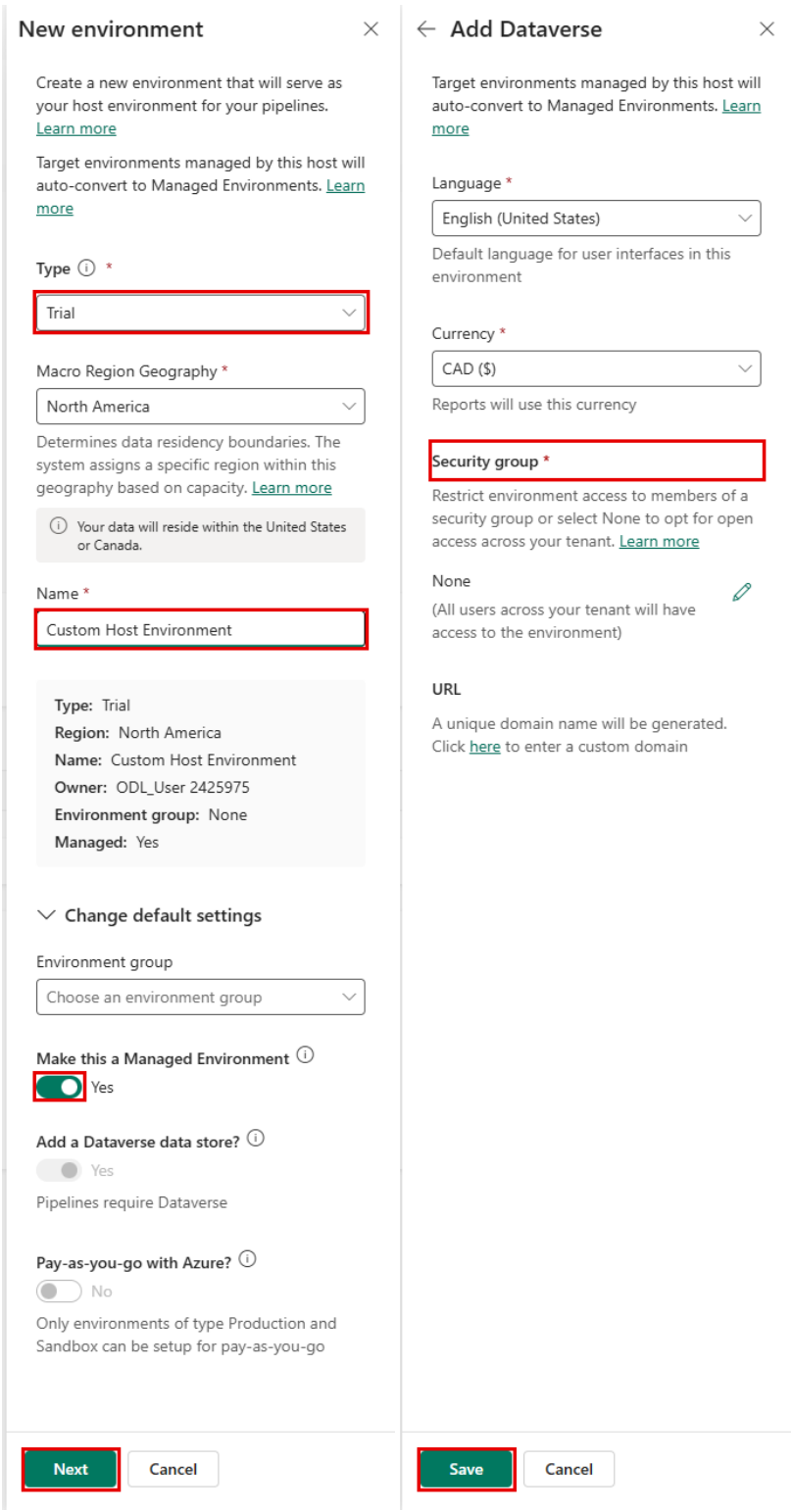
Verify the Pipelines Host environment

Use the existing **Pipelines Host** environment you verified in Lab 1; do not create or rename an environment. This special-purpose environment stores custom pipeline configuration, security, and run history.

1. In the Power Platform admin center, select **Manage** > **Environments** and confirm that **Pipelines Host** exists.
2. Open Power Apps, select **Pipelines Host**, then **Apps**. Confirm that **Deployment Pipeline Configuration** is available.
3. If the environment or app is missing, ask your instructor to complete the custom host setup before continuing.

   Setting up a custom host in your own tenant: In the Power Platform admin center, select **Deployment** > **Overview** > **+ New custom host**. This creates an environment with the Power Platform Pipelines modules pre-installed. Create it as a **Trial** environment, not a Developer environment: each user can have up to 3 Developer environments, and if you pick Developer at that limit, Save keeps loading and the environment is not created.





We will use **Development 1** as the source and **Test** as the target for this custom host pipeline. Keep the environment IDs for these two environments handy:

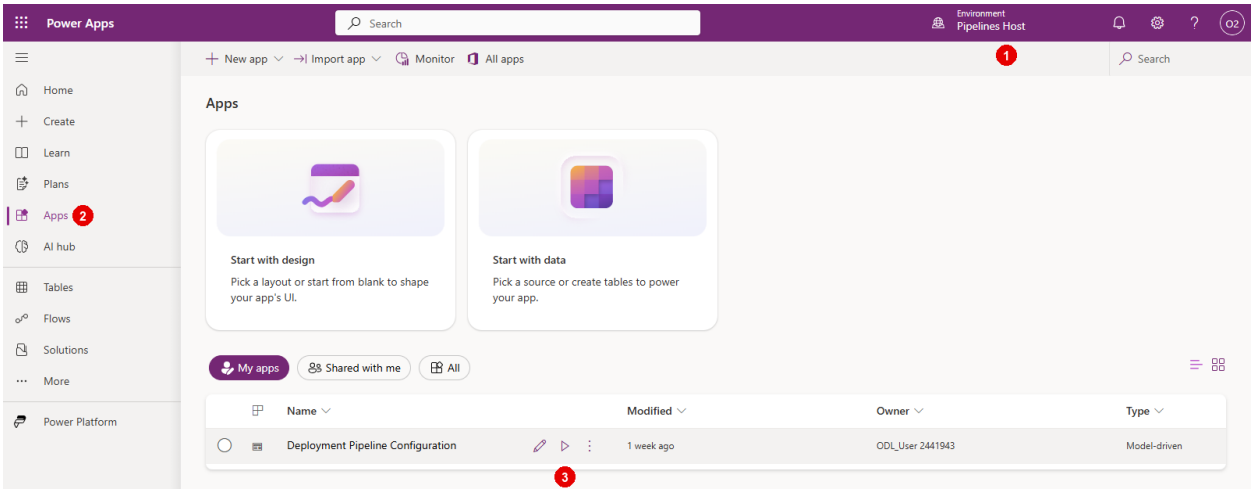
Environment Name: Development 1

Environment Type: Development
Environment ID:

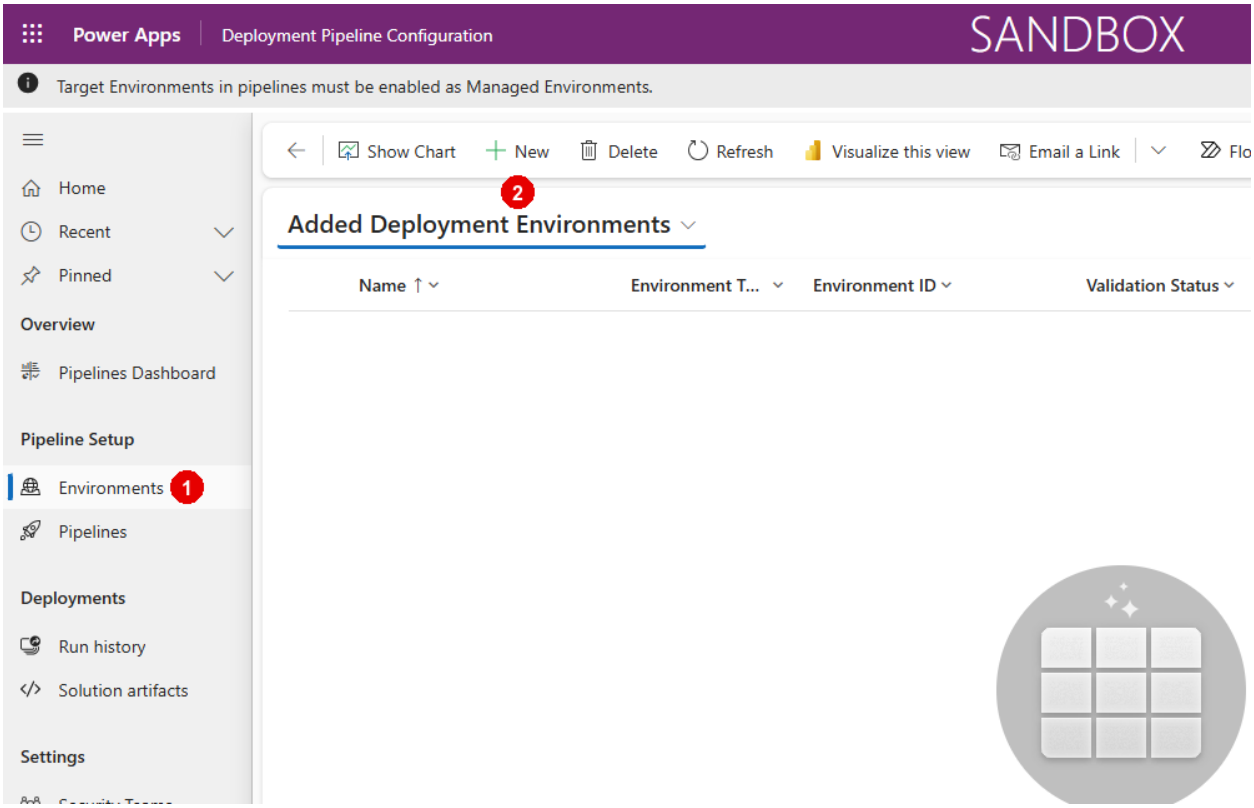
Environment Name: Test
Environment Type: Target
Environment ID:

Configure environment records for the pipeline

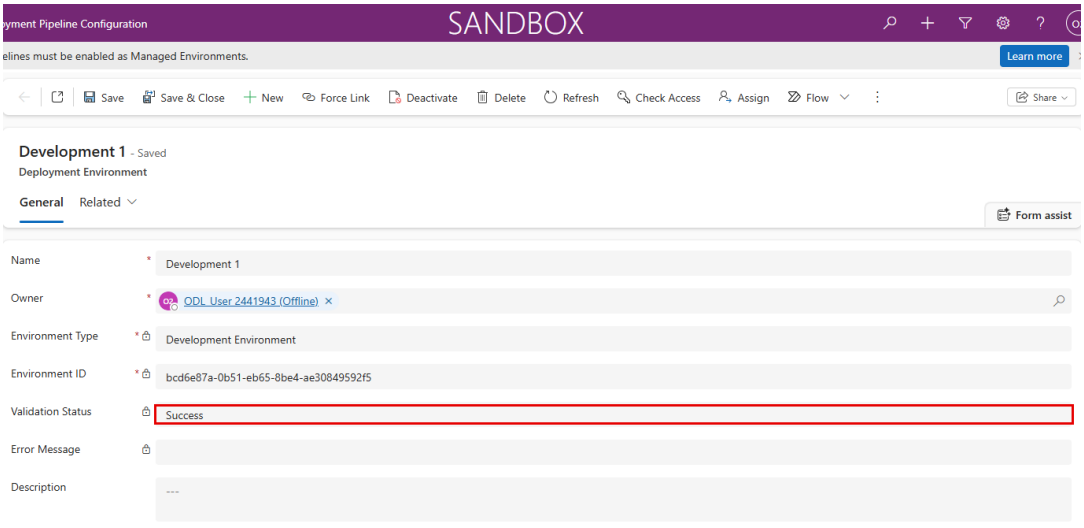
1. Navigate back to PowerApps maker portal at https://make.preview.powerapps.com/
2. In the top right hand corner in the Environment selection dropdown, select **Pipelines Host**
3. Click on the **Apps** tab
4. Click on the **Deployment Pipeline Configuration** app to play it



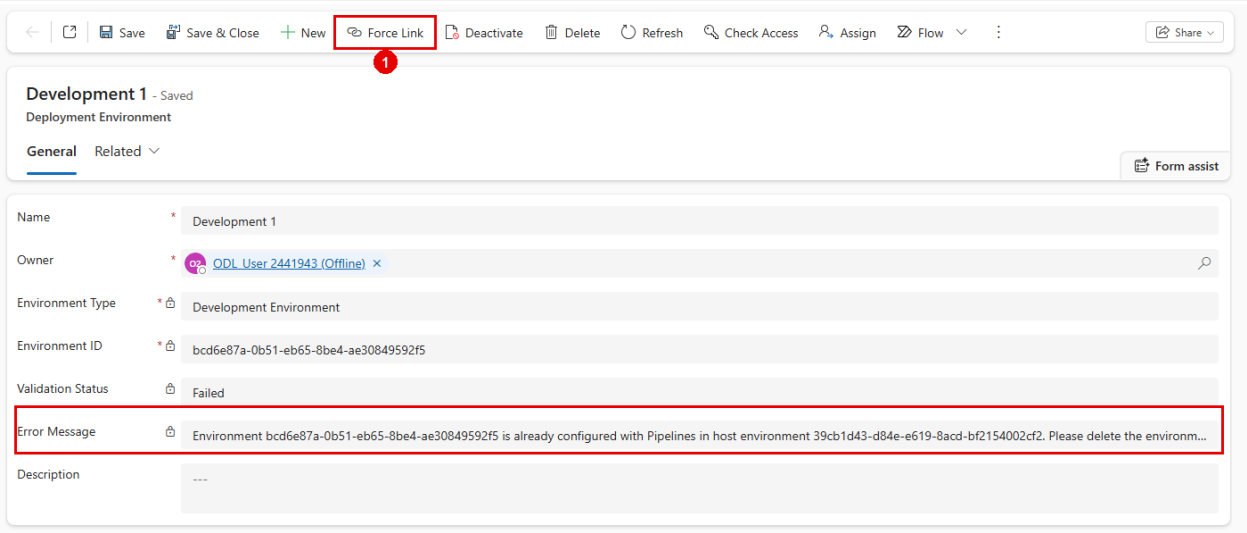
5. In the *Deployment Pipeline Configuration* app, select **Environments** on the left pane

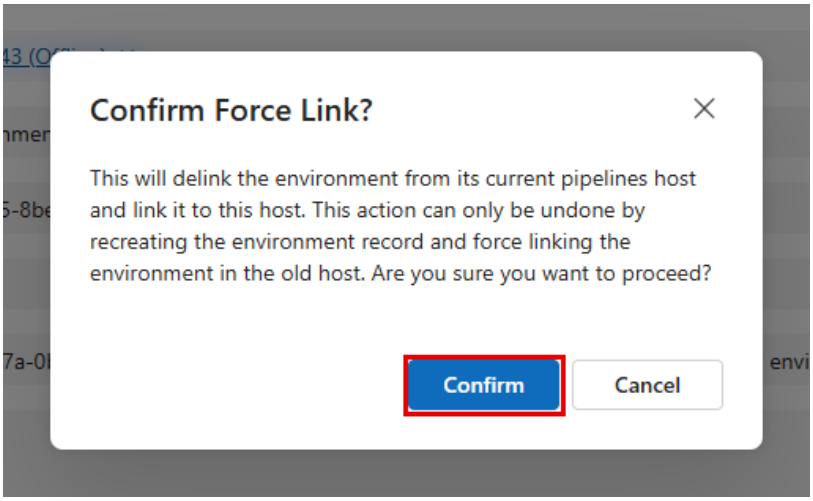


6. Let's create the development (source) environment record for **Development 1**. Select **New** in the ribbon to create a new environment record
    a. **Name:** Development 1
    b. **Environment Type:** Select *Development* for your source environment
    c. **Environment Id:** <Environment ID of Development 1>
    d. Click **Save**
    e. Wait for one minute, and click refresh in the ribbon to verify *Validation Status* equals **Success**

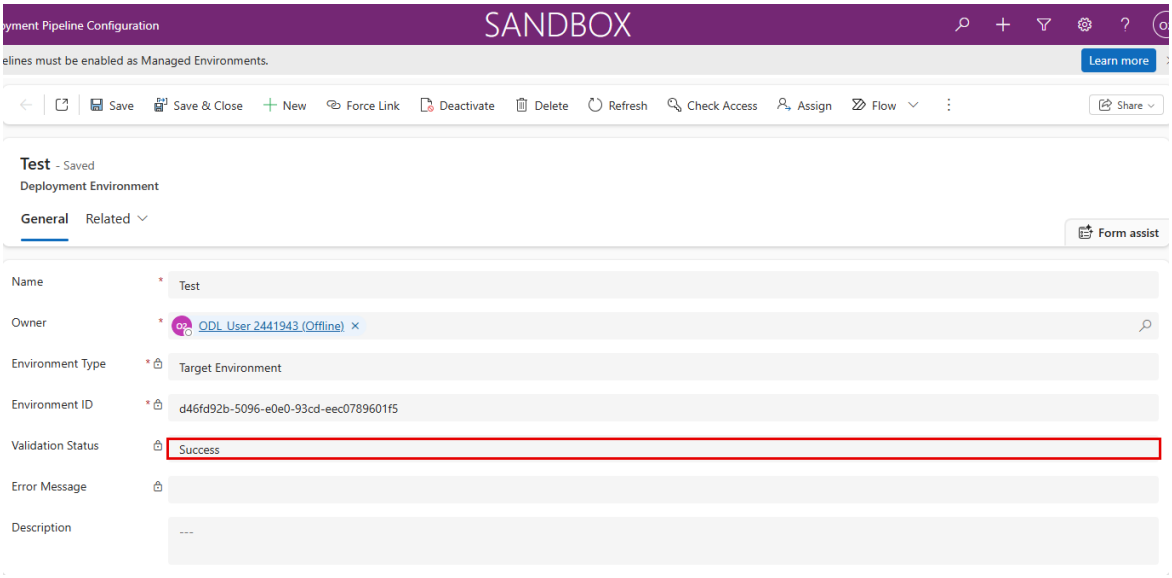


f. Since you already used **Development 1** in your Platform Host pipeline, the validation will fail: the environment is already linked to the platform host. Select **Force Link** in the ribbon, click **Confirm**, wait for one minute, and click **Refresh** until Validation Status equals **Success**.





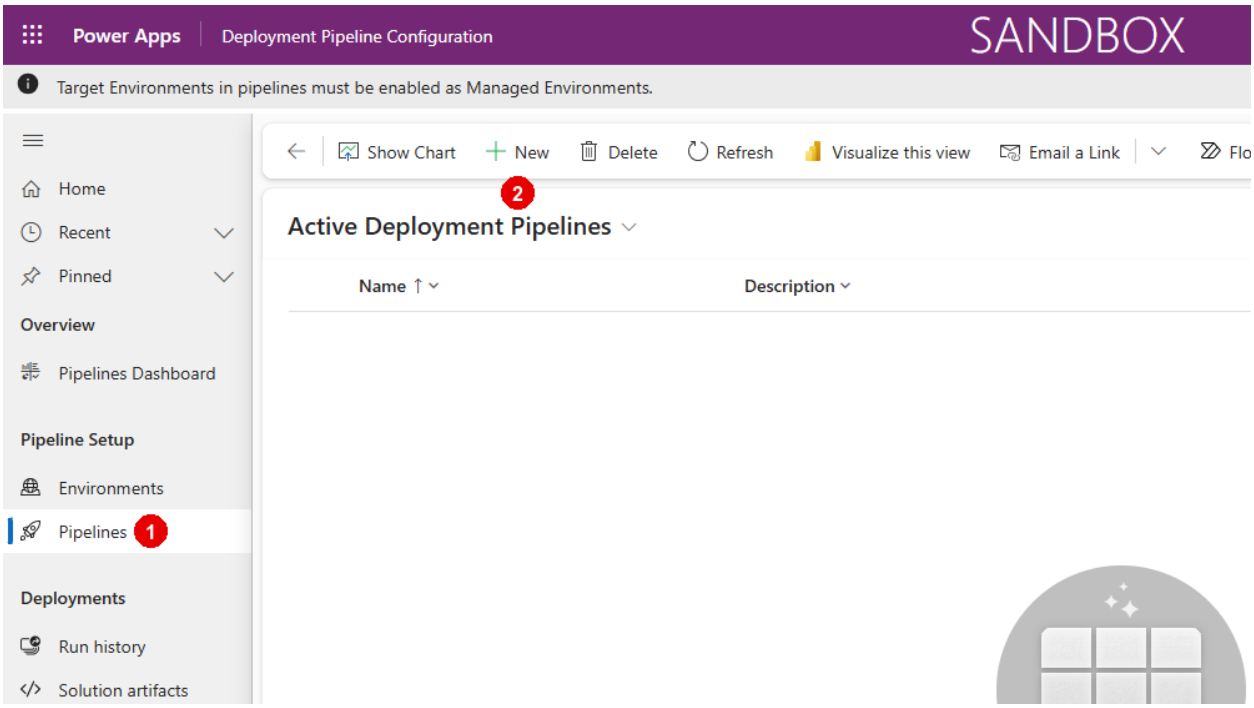
7. Let's create the **Test** (target) environment record. Select **New** in the ribbon to create a new environment record
   a. **Name:** Test
   b. **Environment Type:** Select Target for your target environment
   c. **Environment Id:** <Environment ID of Test>
   d. Click **Save**
   e. Wait for one minute, and click refresh in the ribbon to verify *Validation Status* equals **Success**
   f. Your Platform Host pipeline also deployed to **Test**, so validation fails here too. Use **Force Link** as described above.



8. In the real world, repeat this step for each additional target environment.

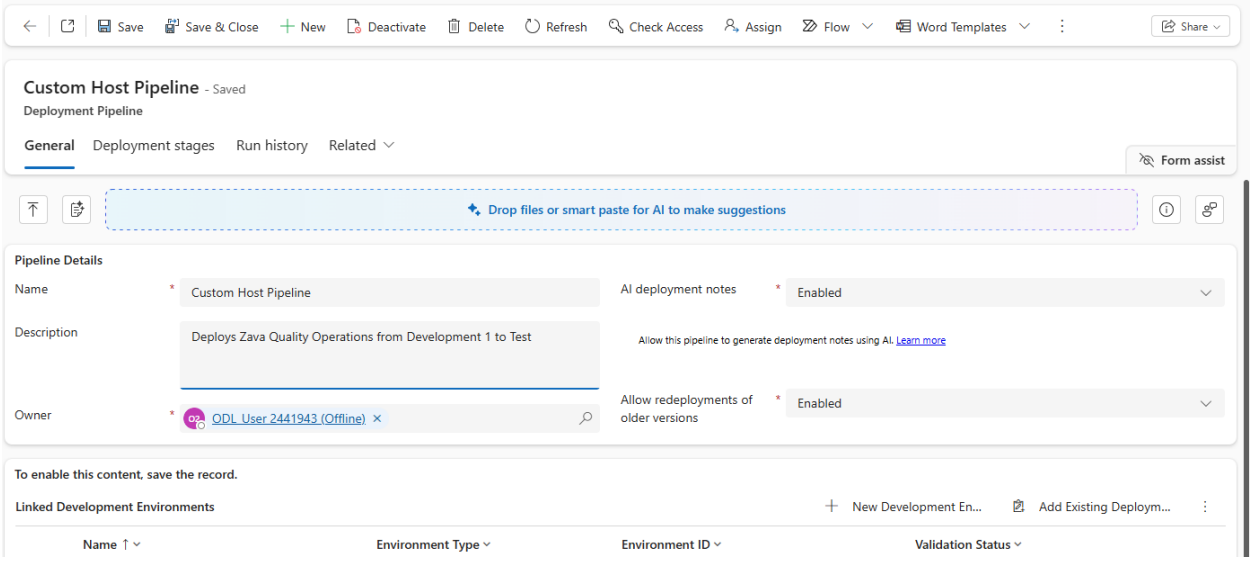
Configure a basic pipeline

9. Select **Pipelines** in the left navigation pane



10. Select **New** in the ribbon to create a new deployment pipeline record
    a. **Name:** Custom Host Pipeline
    b. Description: *optional*, add a description of the purpose this pipeline serves
    c. AI deployment notes: Enabled

d. **Allow redeployments of older versions:** Enabled
e. Click **Save**



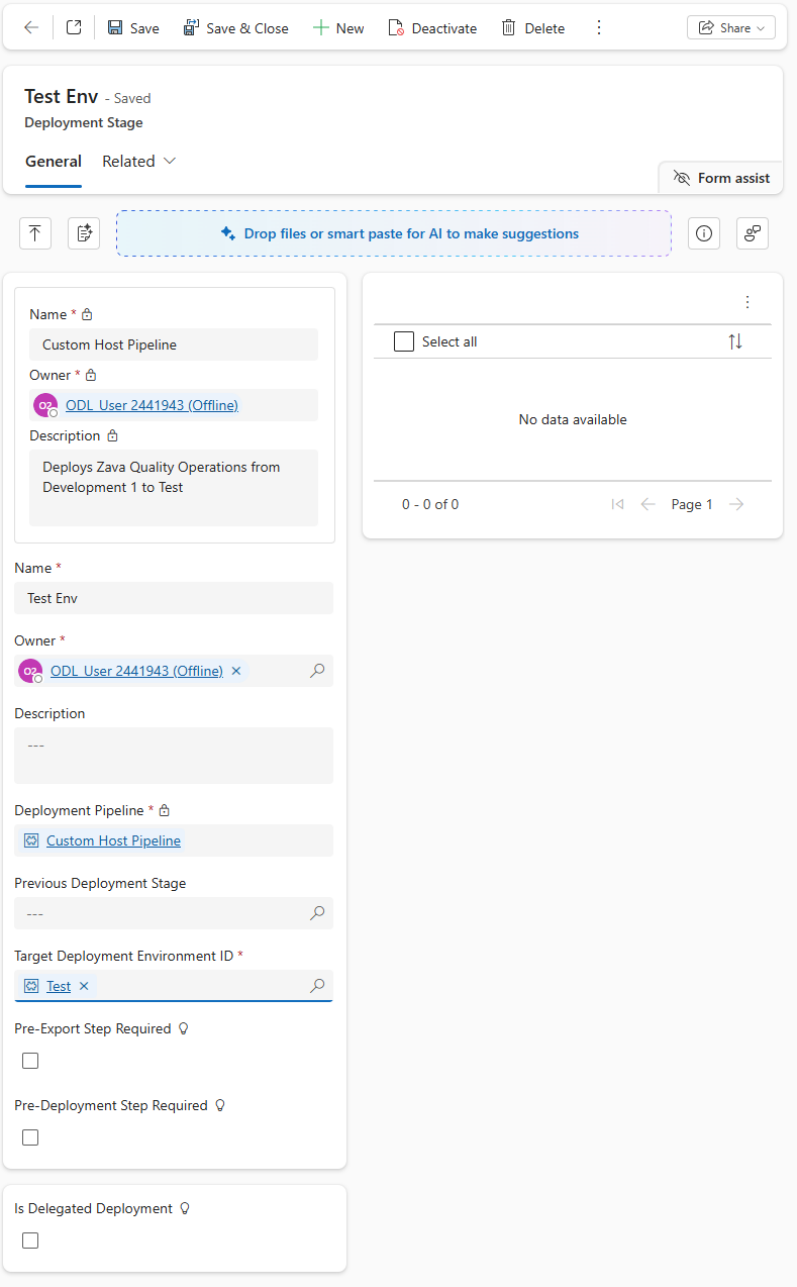
11. Scroll down to the *Linked Development Environments* grid

a. Click on the three vertical dots on the top right corner of the grid to open additional grid options
b. Select **Add Existing Deployment Environment**, then select the development (source) environment you configured earlier in this lab, then click **Add**



12. In the **Deployment Stages (Deployment pipeline)** grid, select **New Deployment Stage**
13. Complete the form:

a. **Name:** Test Env
b. **Description:** *optional*, add a description of the environment you are deploying to
c. **Previous Deployment Stage:** *Leave blank as this will be our first deployment stage*
d. **Target Deployment Environment ID:** Test
e. Click **Save and Close**



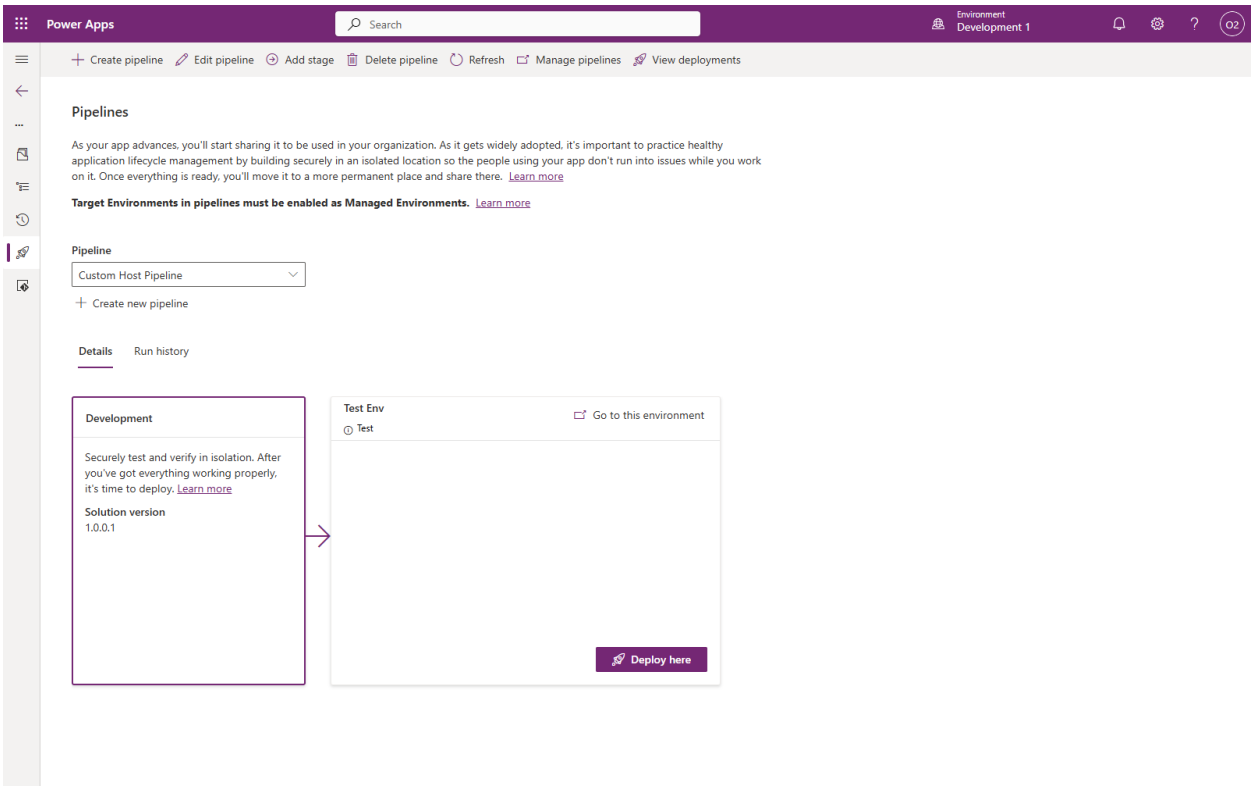
Note: The stage is named **Test Env** so you can tell it apart from the **Test** environment it deploys to. On the Pipelines page, the stage card shows both.

14. In the real world, repeat the previous two steps if you have more than one target environment for your pipeline, with one difference: in the **Previous Deployment Stage**

field, select the stage you configured in the previous step (for example, Test Env). In this workshop, Production is used in Lab 4, so this pipeline has only the Test Env stage.

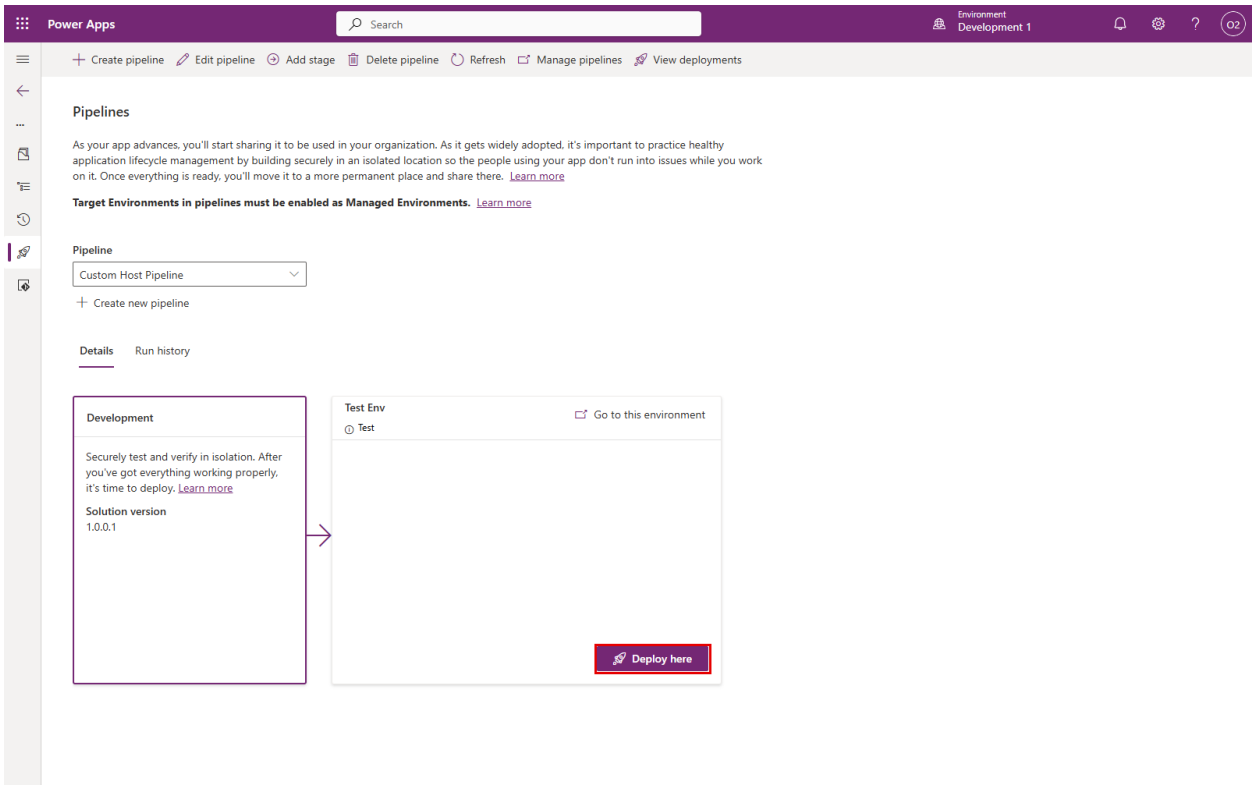
15. This pattern can be repeated to add up to 7 deployment stages in your pipeline.

Congratulations! You have created a basic Custom Host Pipeline. Go to **Development 1** and open **Zava Quality Operations**. Then, click on the **Pipelines tab** (rocket icon) to open the Pipelines pane. Here, you will see your newly created pipeline:



Deploy Using a Custom Host pipeline

1. On the **Test Env** stage, click **Deploy here**.



2. Keep **Now** and click **Next**, review the AI deployment notes, then click **Deploy**.

3. Wait until the **Test Env** stage shows the new version as **Last successfully installed version**.

   Note: You don't need to remove the earlier deployment first. Zava Quality Operations is already in **Test** from Task 1, and deploying again simply upgrades it. You remove it only before running the delegated pipeline later in this lab, so that the Share step appears.

So far, the two types of pipelines we've explored enable basic deployments only. However, some processes call for greater control over the deployment process to ensure production assets are kept safe from unintentional or unapproved changes. For example, while administrators have permission to deploy to production, it's important makers and developers do not have elevated access. Or, maybe the process mandates an admin or change manager's approval before a deployment to production can be initiated. Let's configure our pipeline to support these processes.

Extend Pipelines

Gated extensions

In this section, we will extend deployment pipelines we created in the sections above. Extending pipelines allows organizations to tailor pipelines to their unique needs. For example, you can add approvals, deploy via service principals, and integrate with internal systems of record, Azure DevOps, GitHub, and much more. Custom steps can be initiated at various stages of the pipeline. These steps are defined by Microsoft Dataverse business events. In Power Pipelines terminology, these initiation stages are called "Gated Extensions." Here are the 3 available gated extensions:

Pre-export step required: Allows running custom validation logic when a deployment request is submitted. Pipelines won't export the solution from a development environment until this step is marked as completed. This should only be used on the first stage of the pipeline.

Pre-deployment step required: Allows initiation of custom steps after a deployment has been triggered, but before installation begins on the target environment. This gated extension can be used to trigger processes such as approvals before a solution is deployed to the target environment.

Is delegated deployment: Carries out deployments using a service principal or pipeline stage owner's identity instead of the requesting maker's identity. This identity ensures makers can request deployments without elevated (or any) access within target environments. Requires approval from an authorized identity.

Each step of a pipeline deployment triggers a real-time event at the beginning and completion of the step for which you can initiate custom logic. Additional triggers are produced when gated extensions are enabled. These correspond to the custom step inserted when an extension is enabled on the pipeline stage.

The tables here indicate triggers and actions required for each extension:

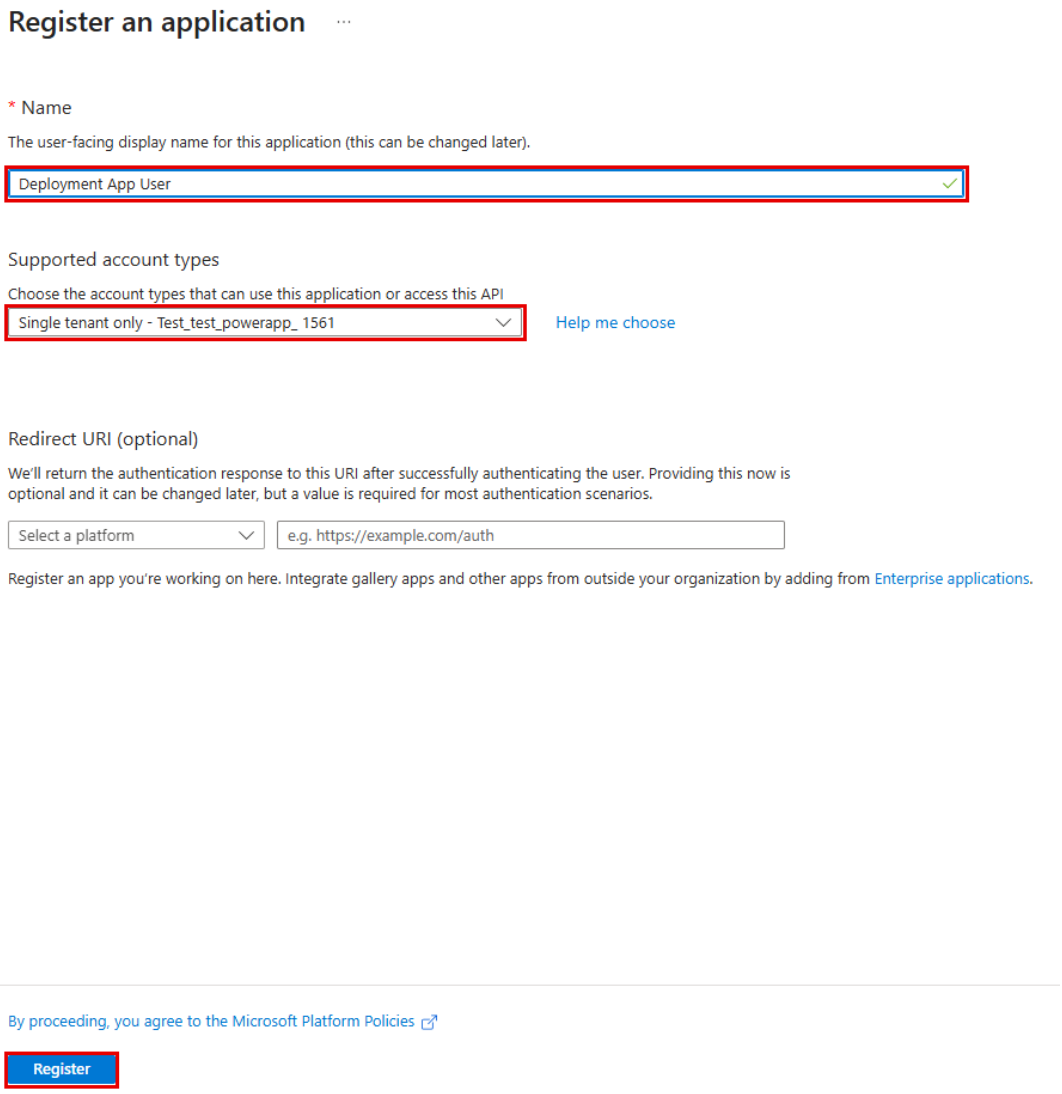
Gated extension	Step started trigger	Step completed trigger	Unbound action	Connection to use
Pre-export step required	OnDeploymentRequested	N/A	UpdatePreExportStepStatus	Any identity with access to update the deployment stage run record
Is delegated deployment	OnApprovalStarted	OnDeploymentCompleted	Create a connection as the service principal or pipeline stage owner as configured on the pipeline stage. The pipeline stage owner must be an owner of the service principal in Microsoft Entra ID.	
Predeployment step required	OnPreDeploymentStarted	OnPreDeploymentCompleted	UpdatePreDeploymentStepStatus	Any identity with access to update the deployment stage run record

Now, let's extend the Custom Host pipeline we created to include an Approval step. Once we get the approval, we will perform a Delegated deployment using an Application user.

Create Application User

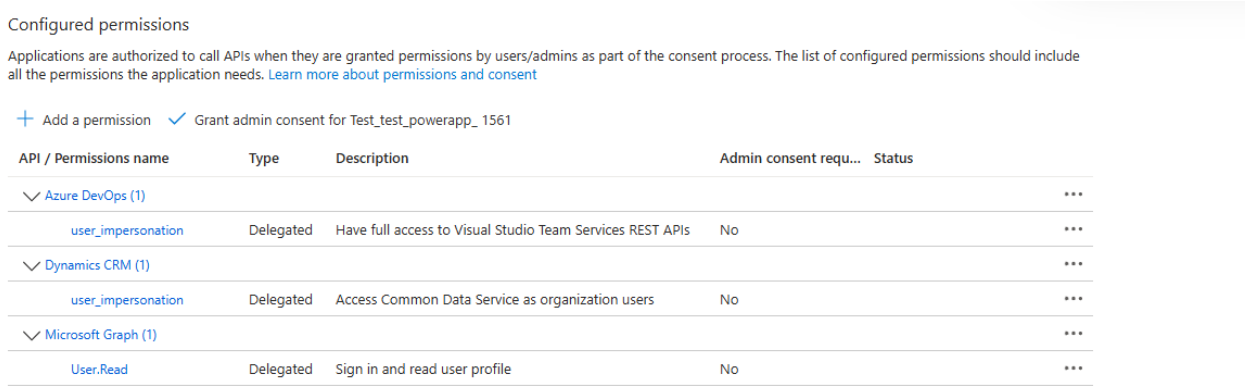
Next, you'll make the **Test Env** stage deploy as a service principal instead of as you (a **delegated deployment**). A service principal is the identity of an app registration in Microsoft Entra ID, so start by registering an app. The following sections give it a client secret, add it as an application user in each environment, and set it on the stage.

1. Navigate to https://portal.azure.com
2. Click on the **Microsoft Entra ID** tile
3. Click on **App Registrations** in the navigation blade > click **+ New registration**
4. *Name:* Deployment App User
5. Supported account types: **Single tenant only** (your lab tenant), the default
6. Click on **Register**



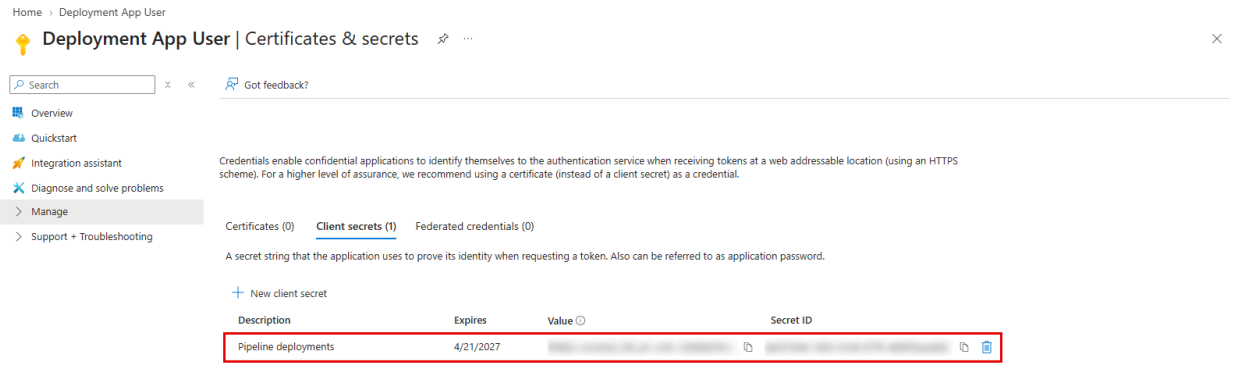
7. Expand the **Manage** tab > click on **API Permissions**
8. Click **+ Add a permission**
    a. In the Request API permissions blade, select the following and grant **user_impersonation** permissions:
        i. Azure DevOps
        ii. Dynamics CRM

The configured permissions should looks like this:

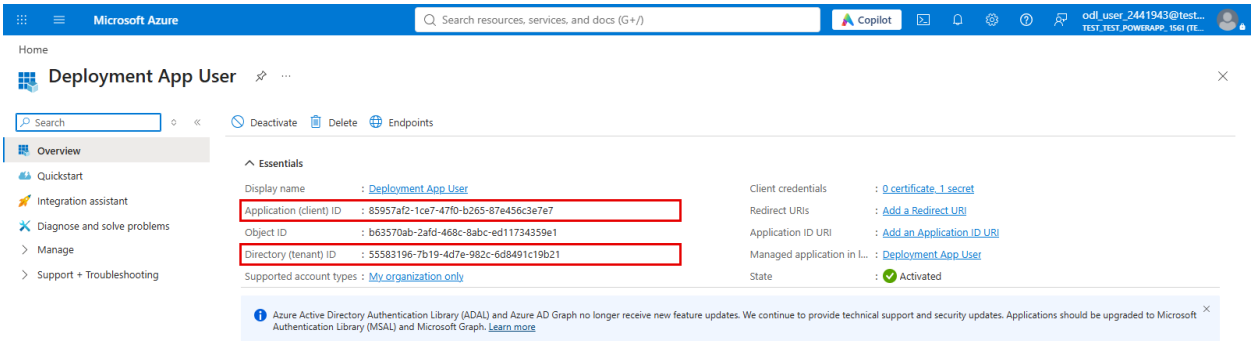


Next, let's add a certificate to our new app user. This is how we will authenticate our connection.

9. Click on **Certificates & secrets** in the navigation blade
10. Click **+ New client secret**
11. *Description:* add a descriptive note
12. *Expires:* Recommended: 180 days (6 months)
13. IMPORTANT: note the **client secret value** before navigating away and store it in an easily accessible place. Tip—use Notepad, stickynotes, or OneNote for easy copy/pate later
    i. Value:

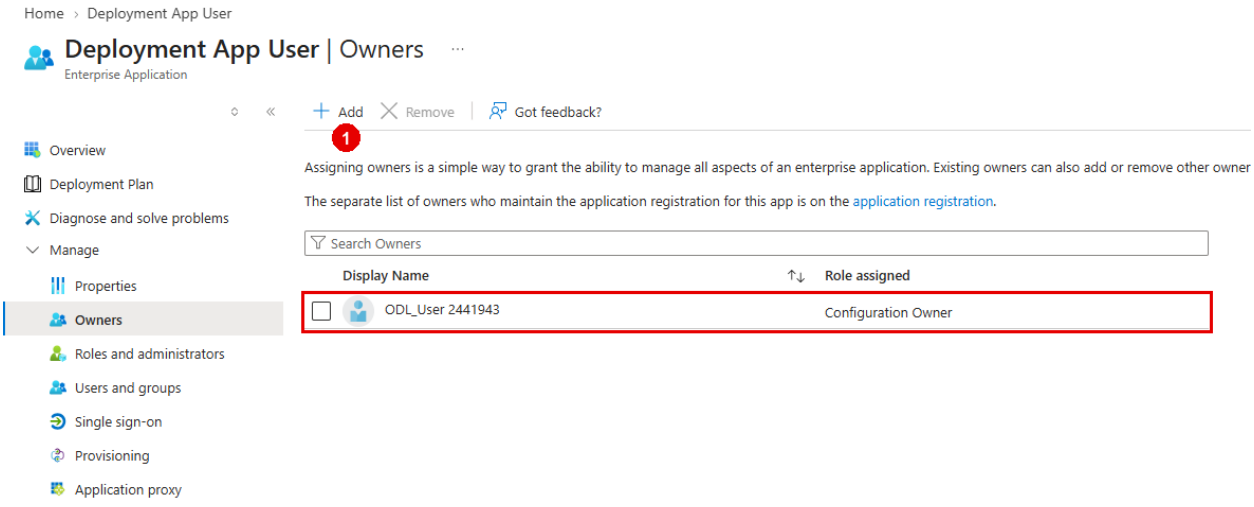


14. Click on the **Overview** tab in the navigation blade
15. Note down **Application (client) ID** and **Directory (tenant) ID**
    i. Application (client) ID:
    ii. Directory (tenant) ID:



As the pipeline stage owner, you must own the app's **enterprise application** (its service principal). Otherwise, saving the delegated deployment stage later in this lab fails.

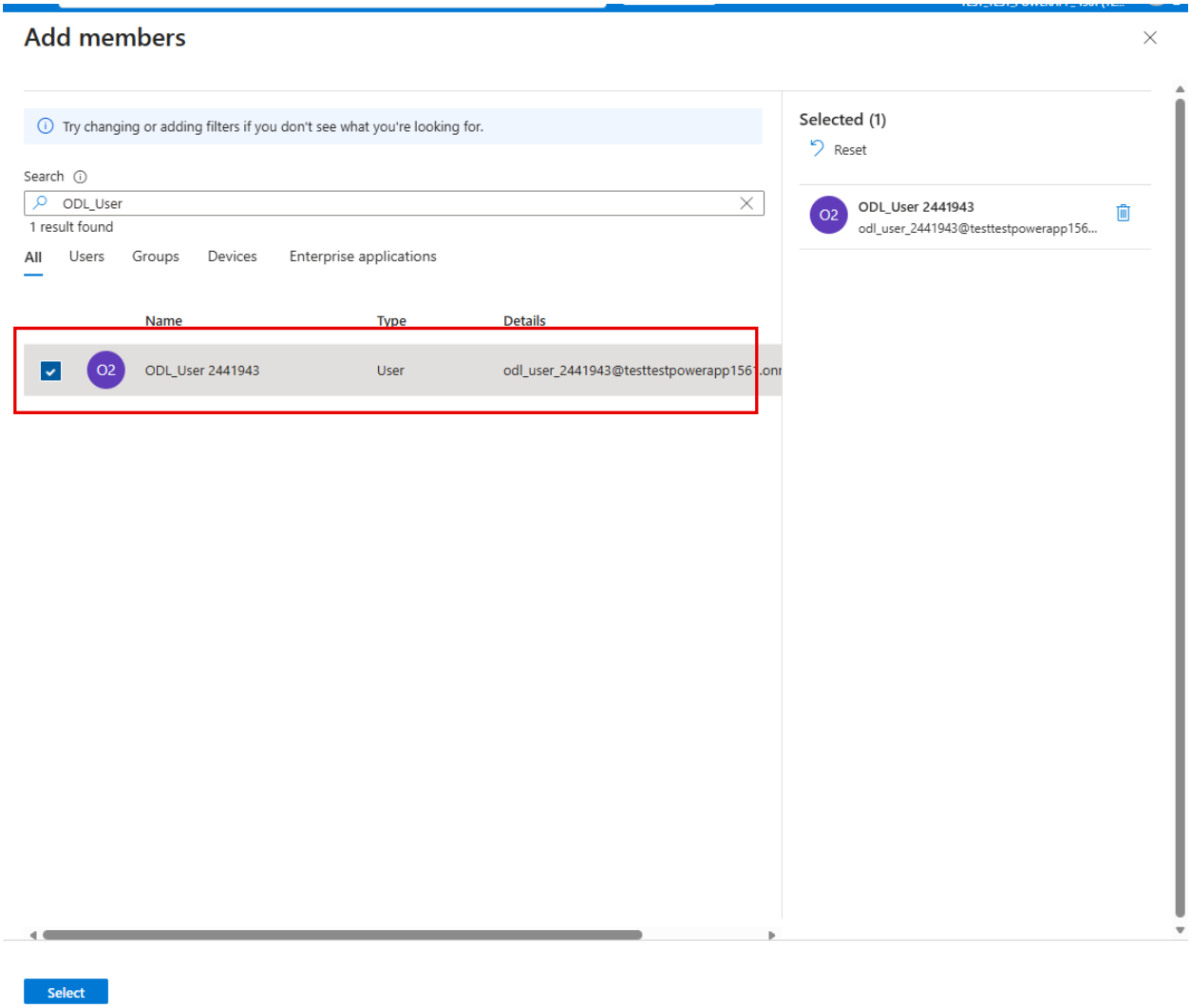
16. On the app registration's **Overview**, click the link under **Managed application in local directory** (Deployment App User). The enterprise application opens.
17. Under **Manage**, click **Owners** > **+ Add**.
18. Search for your lab user, select it, and click **Select**.



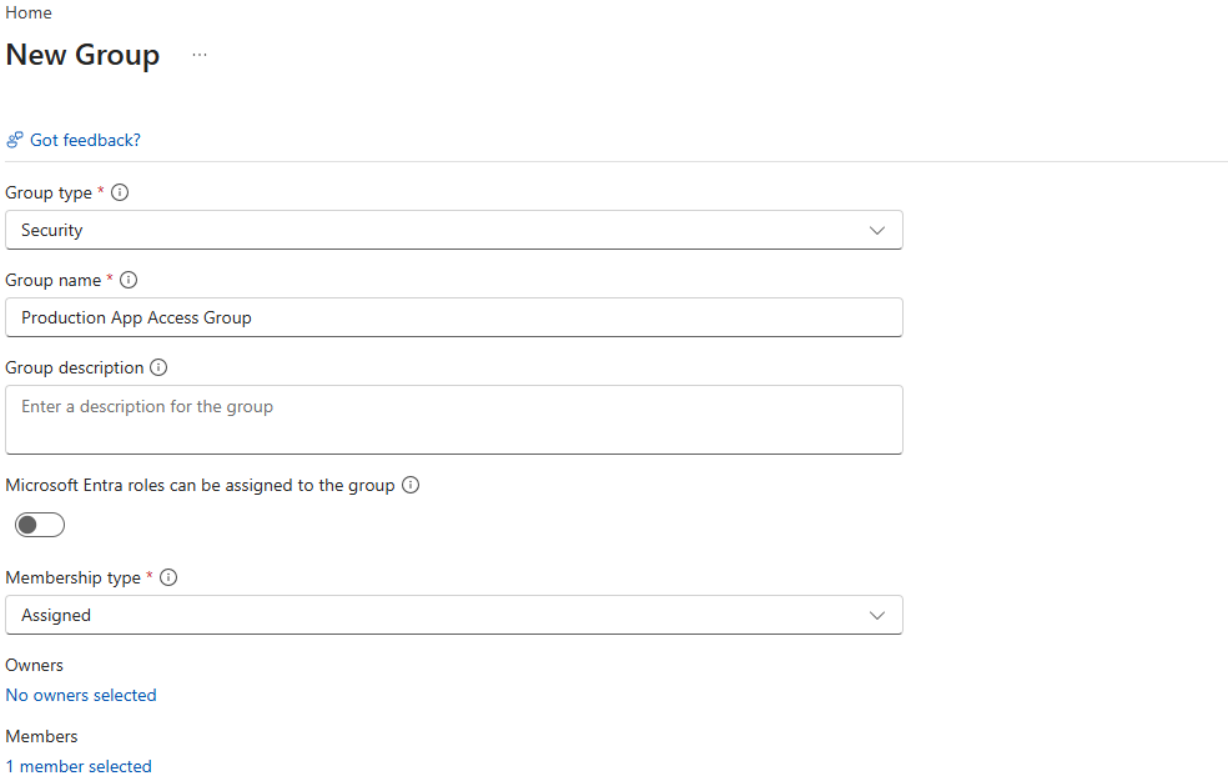
Create Sharing Security Group

While we are still on the Azure portal, let's create our security group.

19. Navigate to the new group creation page by using this link: https://portal.azure.com/?l=en.en-ie#view/Microsoft_AAD_IAM/AddGroupBlade
20. Use the following group details:
    i. **Group Type:** Security
    ii. **Group Name:** Production App Access Group
21. Click *No members selected* to add members to the group
    i. Search for your own lab user, click **Select**


Add members
Try changing or adding filters if you don't see what you're looking for.
Search
ODL_User
1 result found
All
Users
Groups
Devices
Enterprise applications
Name
Type
Details
O2
ODL_User 2441943
User
odl_user_2441943@testtestpowerapp156...
Selected (1)
Reset
O2
ODL_User 2441943
odl_user_2441943@testtestpowerapp156...
Select

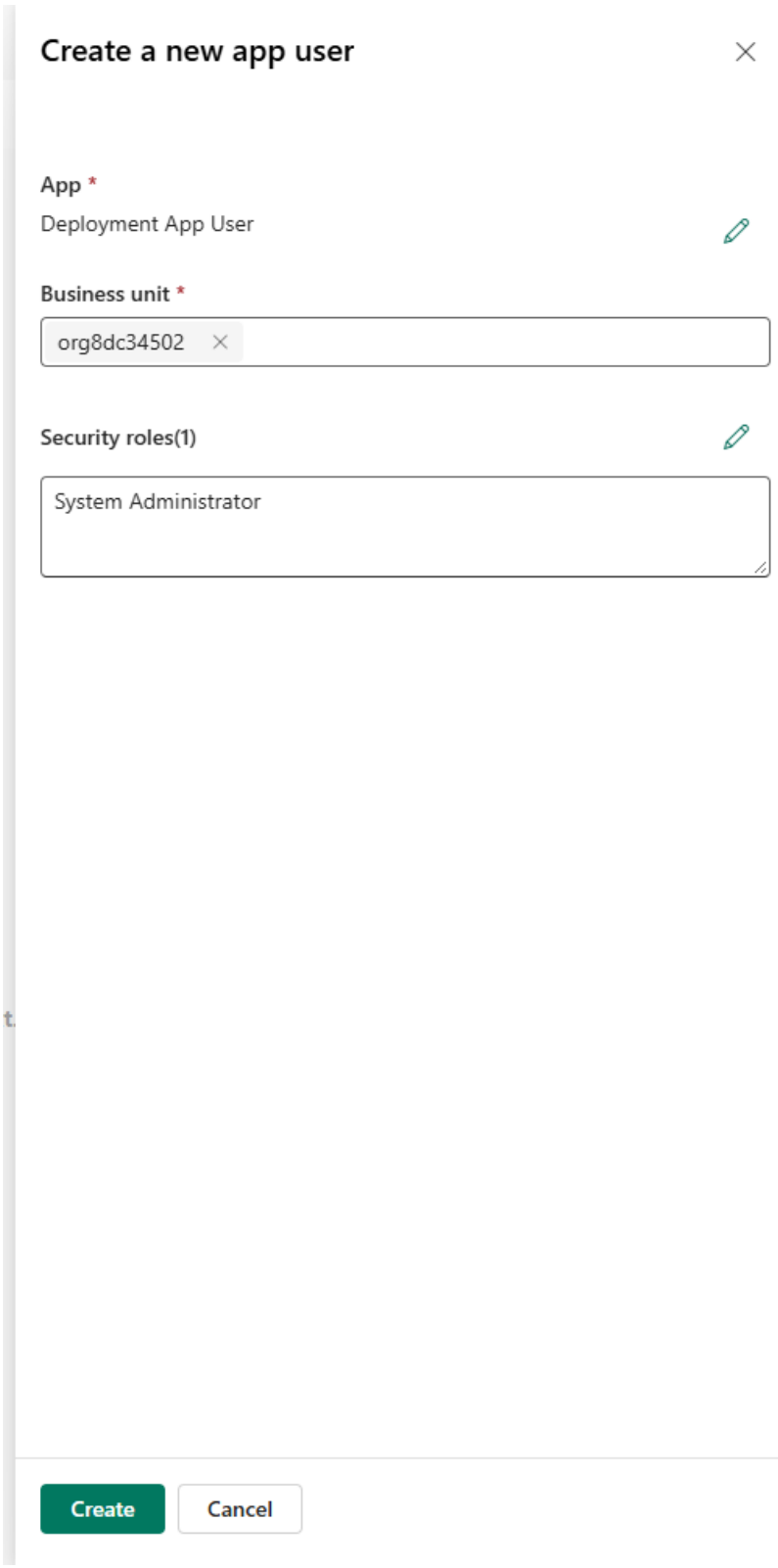
Your group should look like this before clicking **Create**:



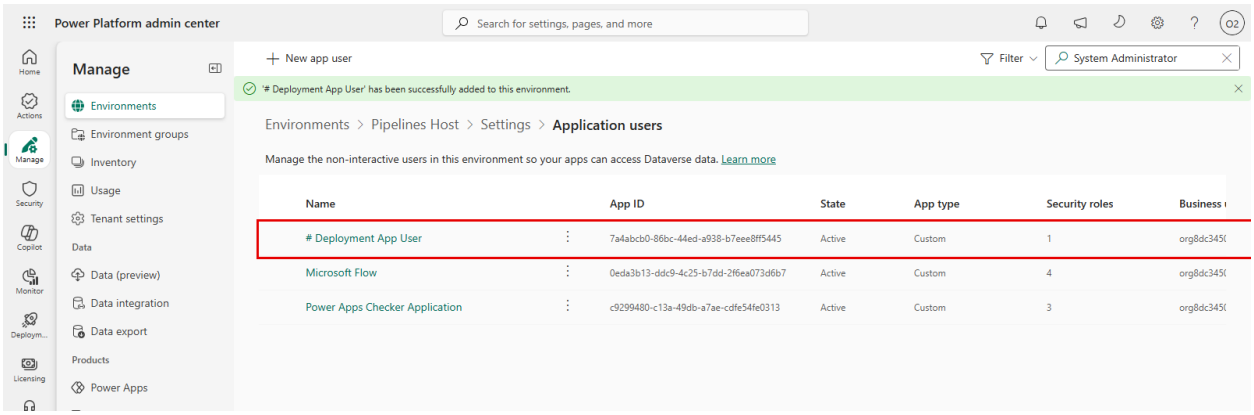
Create application user in Dataverse

Add Application User to each environment of our pipeline, including **Pipelines Host**

22. Navigate to PPAC at admin.powerplatform.microsoft.com
23. In the Environments tab, select **Pipelines Host**
24. In the Access section, under S2S apps, click **See all**
25. Click **+ New app user**
26. App: click **+ Add an app** > select the app registration you created in the Create Application User section (you may need to search the name of the App Registration if it is not listed) > click **Add**
27. *Business unit:* select default BU (type “org” and select the first item)
28. *Security roles:* System Administrator



29. Click **Create**
30. Your application user will be listed with a # prefix in the Name column



2. Repeat these steps for **Development 1** and **Test**

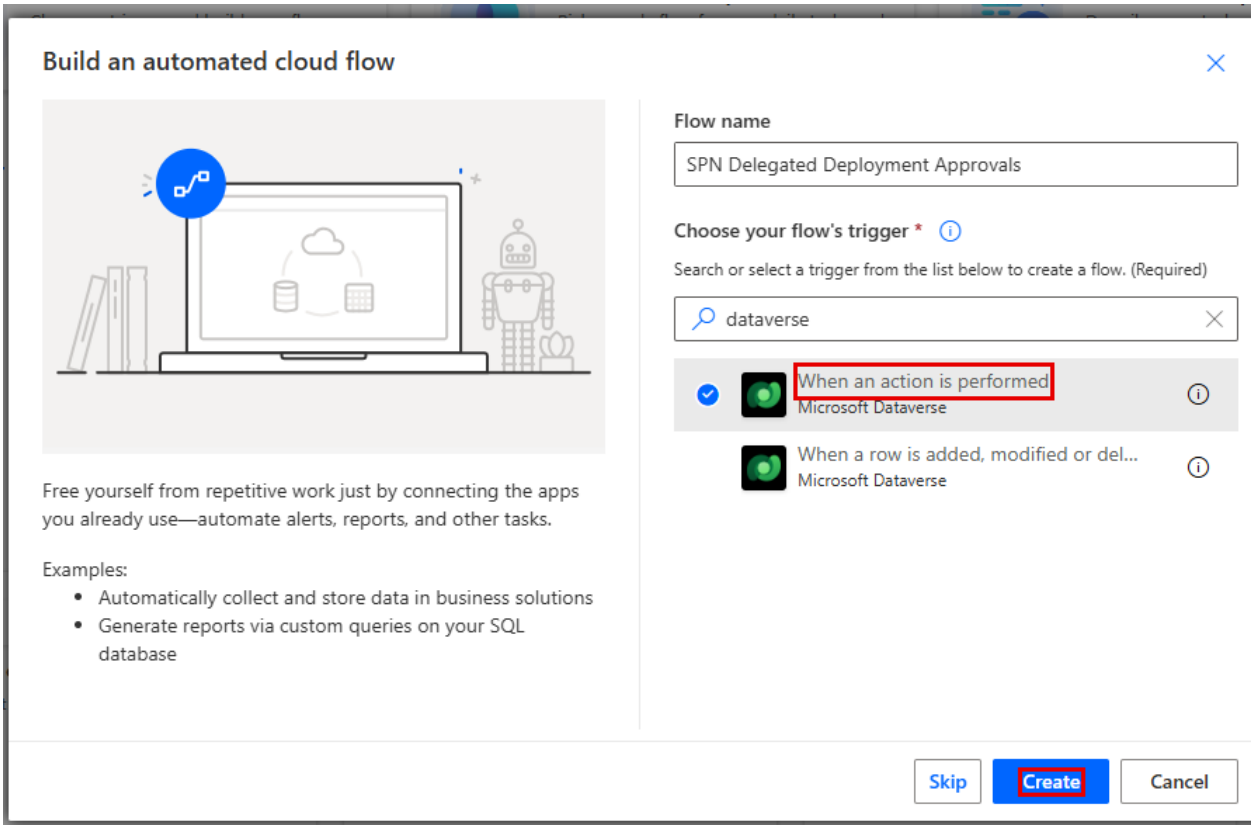
Create an approval + SPN delegated deployment flow

Pathway 1 (recommended) -- Import the ready-made **Lab 3 Approval Delegation Flow** solution. Download Lab3ApprovalDelegationFlow_1_0_0_1.zip from GitHub and import it into **Pipelines Host** (**Solutions** > **Import solution**). If the wizard asks for connections, pick any available ones; you'll switch the Dataverse connection to the service principal next. After import, jump to the **Update Flow after solution import** section.

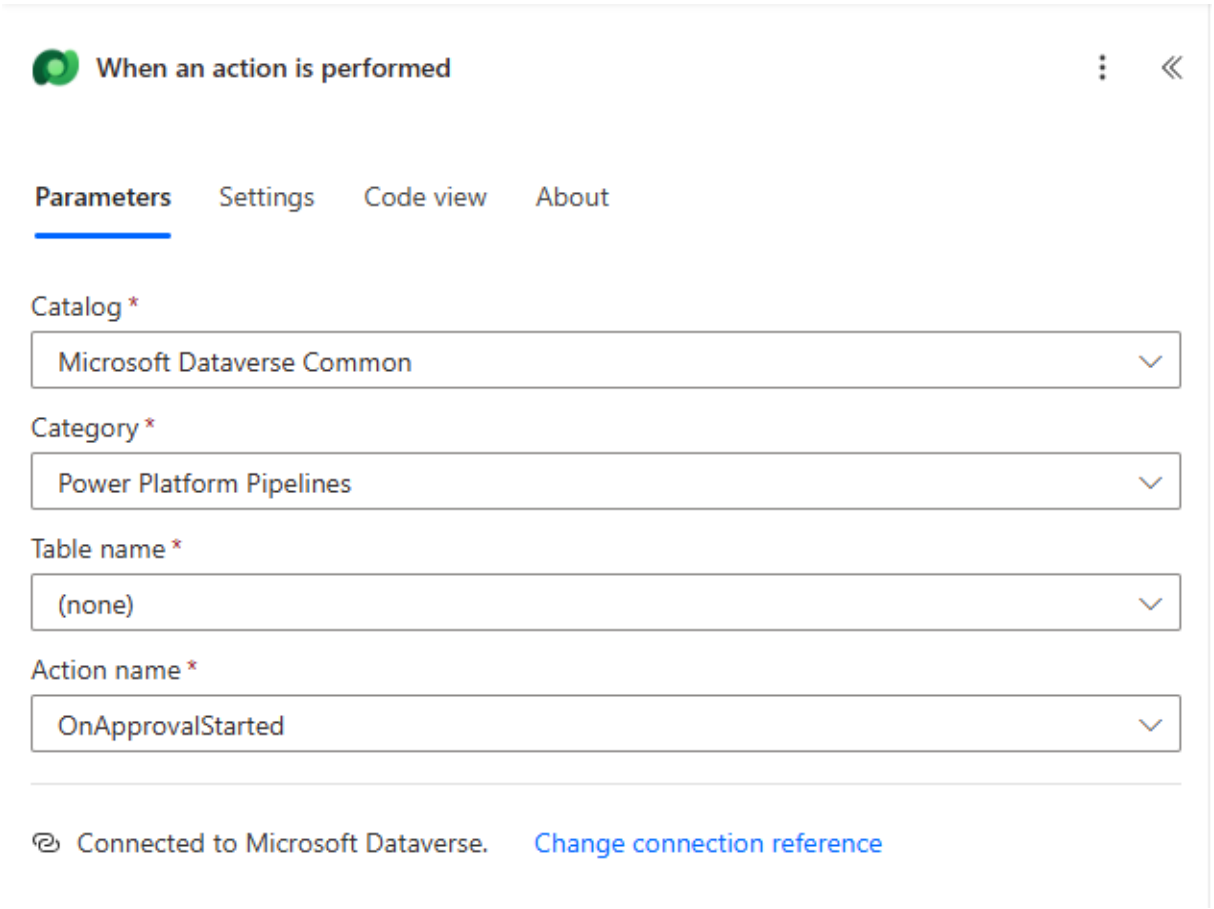
Pathway 2 -- If you would like to create the flow manually, follow the steps below.

NOTE: In the instructions below, we have provided the written expression for the dynamic values. You may choose to copy/paste the expressions from the instructions below, or use the lightning bolt icon to add the dynamic values to your flow manually.

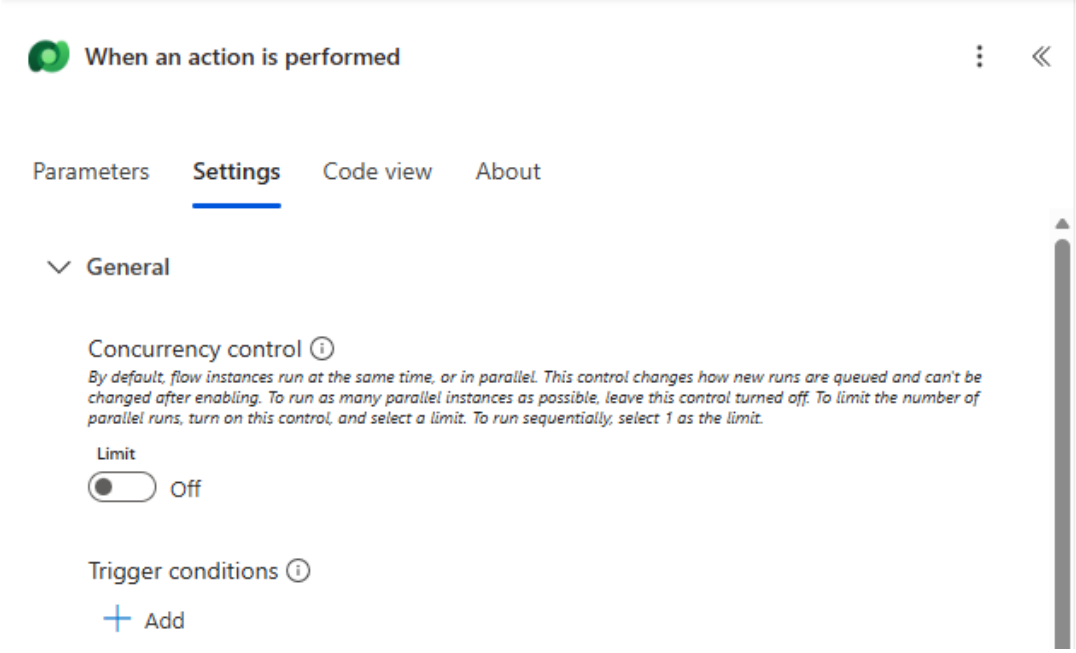
1. Navigate back to https://make.powerapps.com/ and select **Pipelines Host** from the environment picker dropdown
2. Click **Flows** from the left-hand navigation blade
3. Click **+ New flow** > choose "**Automated flow"** and name your flow "SPN Delegated Deployment Approvals"
4. Select the trigger **"When an action is performed"** from the Dataverse connector.



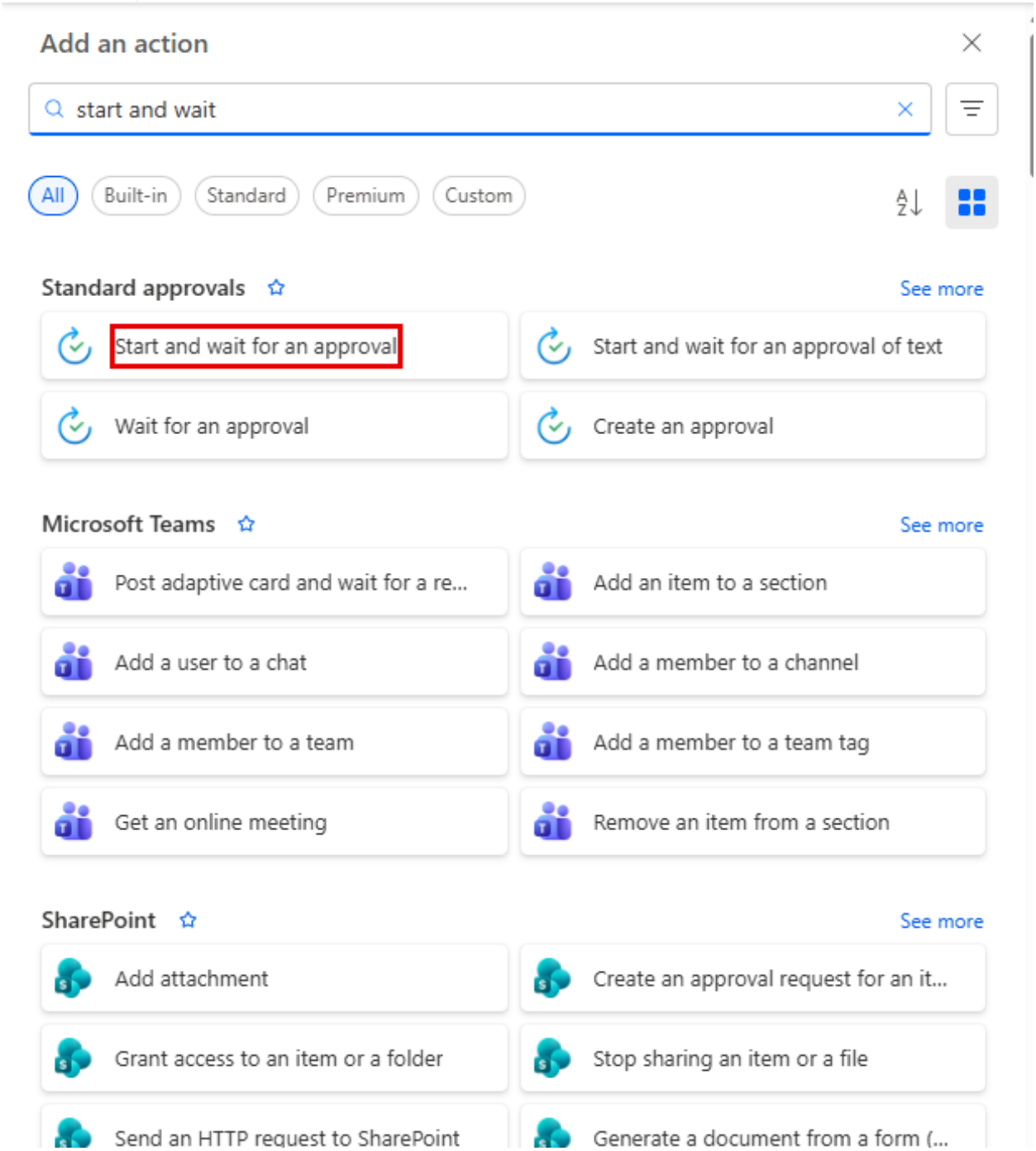
5. Configure the trigger with the following parameters:
    a. **Catalog:** `Microsoft Dataverse Common`
    b. **Category:** `Power Platform Pipelines`
    c. **Table Name:** (none)
    d. **Action Name:** `OnApprovalStarted`



6. Click the three dots and select **Settings**, set the trigger condition to the below parameter. This trigger condition allows us to have a unique flow process for each pipeline. If you'd like to use the same flow for all your pipelines, skip this step:
`@equals(triggerOutputs()?['body/OutputParameters/DeploymentPipelineName'], '<<NAME YOU PROVIDED IN STEP 15.a IN THE PREVIOUS SECTION>>')`



7. Click +New Step and search for **"Start and wait for an approval"** in the Standard Approvals connector.



8. Configure the approval with the following parameters (you may copy/paste the parameters):
    a. **Approval Type:** Approve/Reject – First to respond
    b. **Title:** Deployment to @{triggerOutputs()?['body/OutputParameters/DeploymentStageName']} requires approval
    c. **Assigned To:** <<YOUR EMAIL HERE>>

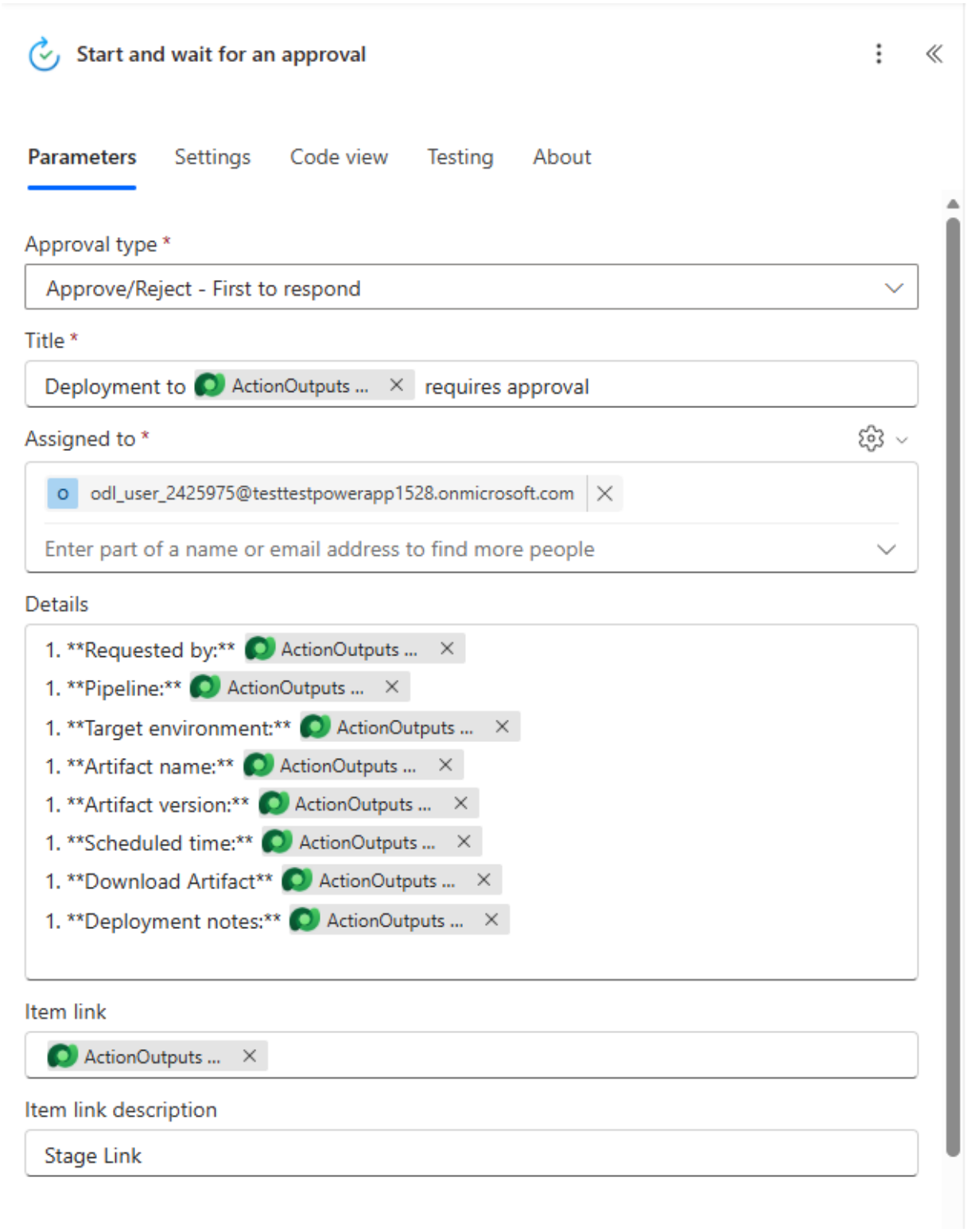
d. **Details:** Provide the deployment details using the provided markdown format. A sample is provided below:

```
1. **Requested by:**
@{triggerOutputs()?['body/OutputParameters/DeployAsUser']}
1. **Pipeline:**
@{triggerOutputs()?['body/OutputParameters/DeploymentPipelineName']}
1. **Target environment:**
@{triggerOutputs()?['body/OutputParameters/DeploymentStageName']}
1. **Artifact name:**
@{triggerOutputs()?['body/OutputParameters/ArtifactName']}
1. **Artifact version:**
@{triggerOutputs()?['body/OutputParameters/SolutionArtifactVersion']}
1. **Scheduled time:**
@{triggerOutputs()?['body/OutputParameters/ScheduledTime']}
1. **Download Artifact**
@{triggerOutputs()?['body/OutputParameters/UnmanagedArtifactFileDownloadLink']}
1. **Deployment notes:**
@{triggerOutputs()?['body/OutputParameters/AIGeneratedSolutionDescription']}
```

e. **Item Link:**
`@{triggerOutputs()?['body/OutputParameters/StageRunDetailsLink']}`

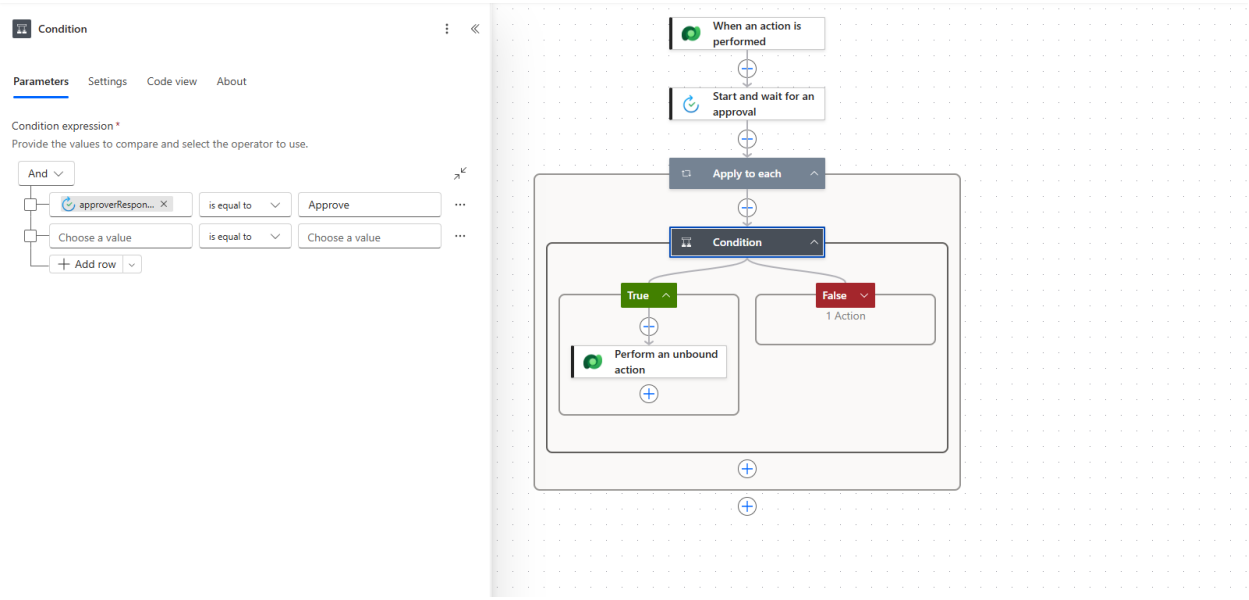
f. **Item Link Description:** Stage Link

9. Click **Save draft**



10. Click + New step and add a **condition** to check and add “responses approver response” dynamic variable to the condition. This is the expression parameter: “if `@items('Apply_to_each')?['approverResponse']` is equal to **Approve**. When you

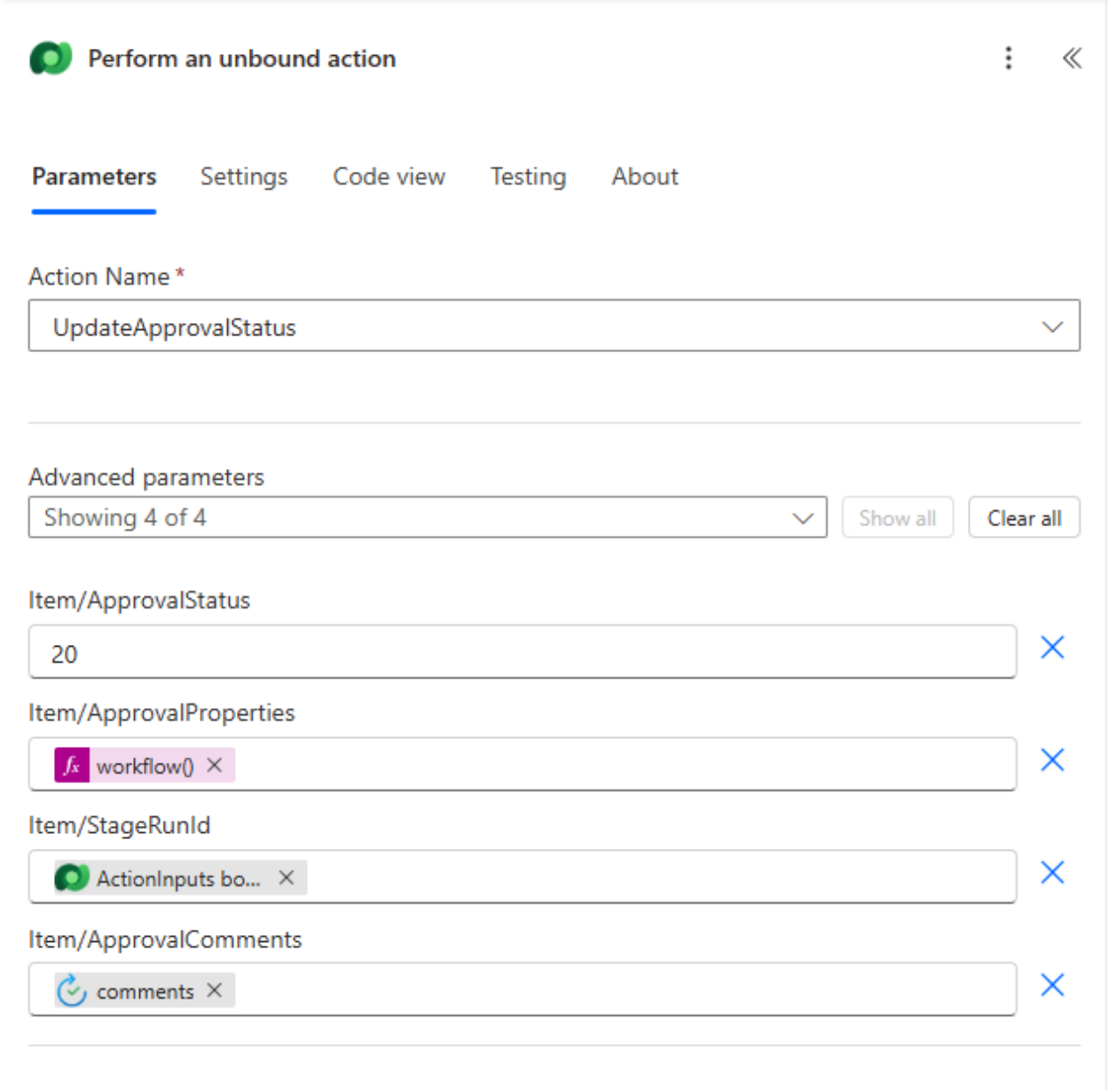
add the Condition parameter, an *Apply to each* will automatically be added around your condition



11. In the *If yes* path, add the **Perform an unbound action** step from the Microsoft Dataverse connector.

12. Configure the true condition with the following parameters:
    a. **Action Name:** `UpdateApprovalStatus`
    b. **StageRunId:** `@triggerOutputs()?['body/InputParameters/StageRunId']`
    c. **ApprovalProperties:** `@{workflow()}`
    d. **ApprovalStatus:** `20`

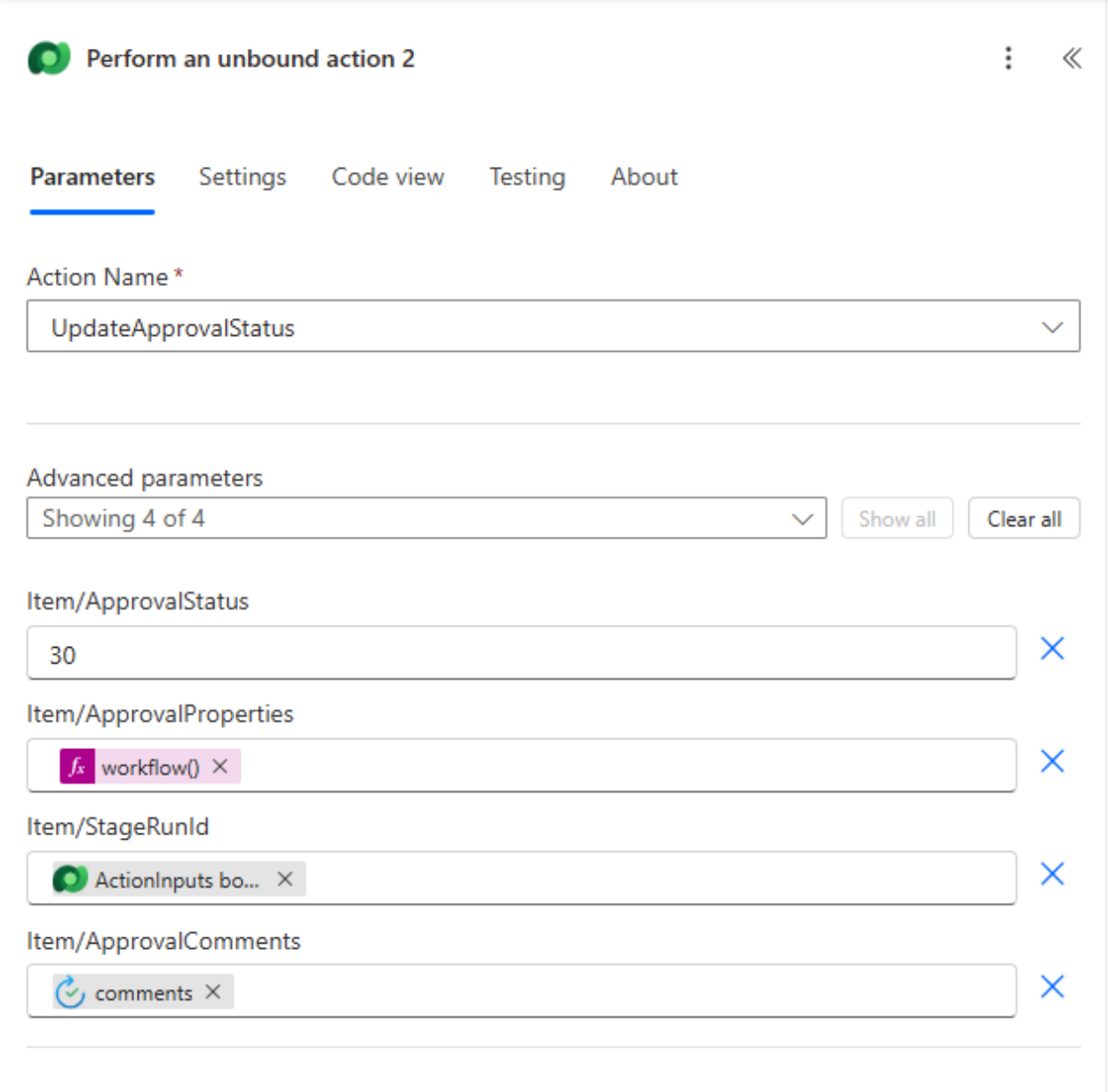
    Tip: *Approval Status Step* mappings: Pending- **10**, Completed – **20**, Failed – **30**



13. If the *If no* path, add the **Perform an unbound action** step from the Microsoft Dataverse connector.

14. Configure the false condition with the following parameters:
    a. **Action Name:** `UpdateApprovalStatus`
    b. **StageRunId:** `@triggerOutputs()?['body/InputParameters/StageRunId']`
    c. **ApprovalProperties:** `@{workflow()}`
    d. **ApprovalStatus:** `30`

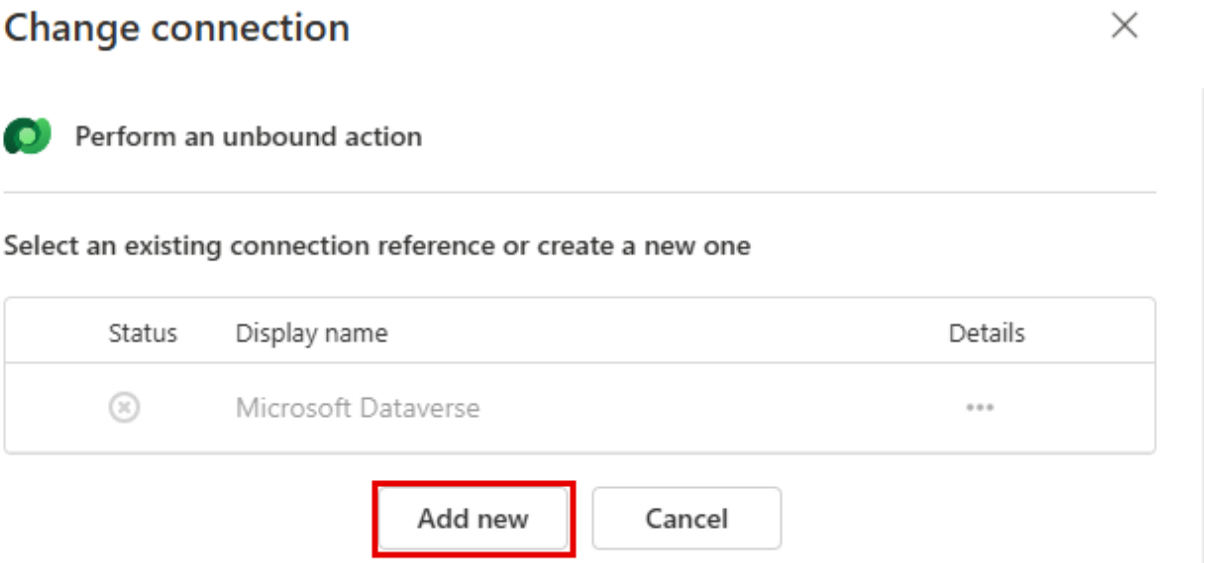
    Tip: *Approval Status Step* mappings: Pending- **10**, Completed – **20**, Failed – **30**



15. Click **Save draft** to save your flow.

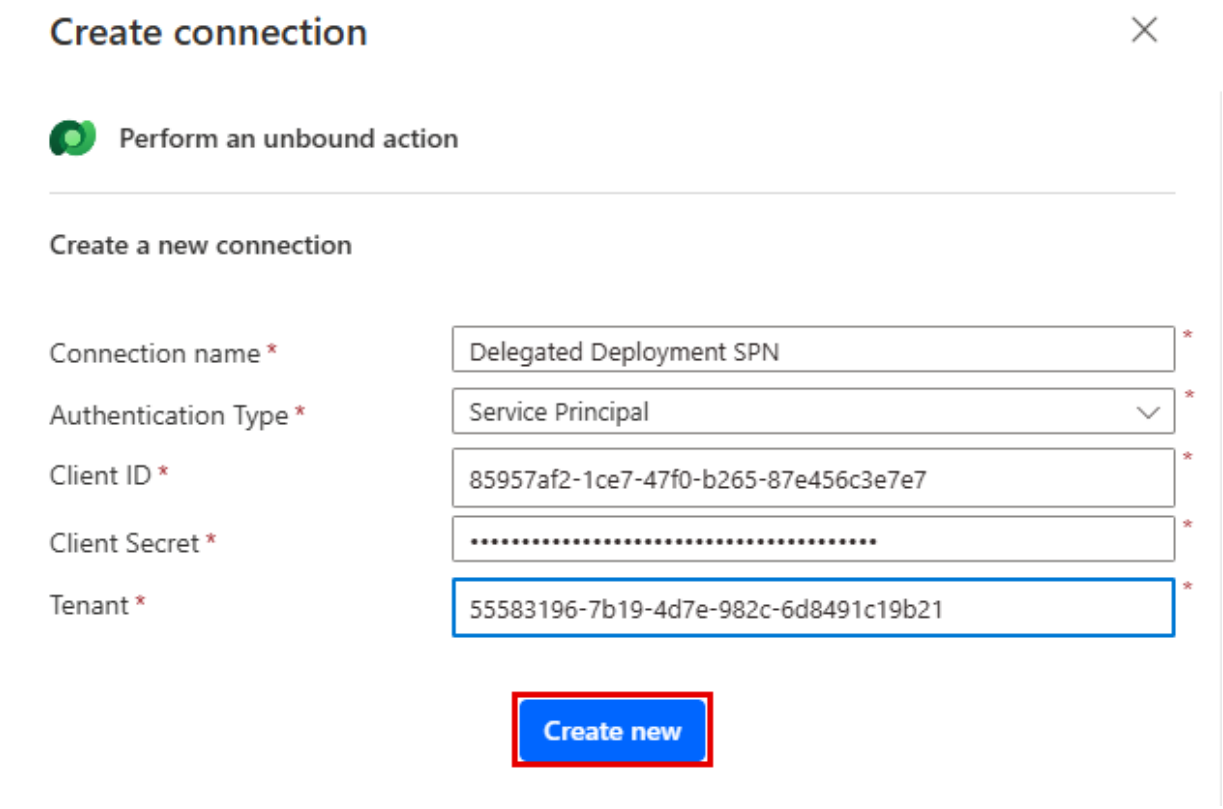
We're not done yet! Since we will be doing a delegated deployment, we need to configure the flow to use the SPN credentials to execute the approval action using the SPN's connection.

16. Select the **If yes** Dataverse action (Perform an unbound action). At the bottom of the Parameters pane, click **Change connection** (**Change connection reference** in a solution flow), then click **Add new**



17. Configure the connection:
    a. Authentication Type: **Service Principal**
    b. Client ID: <Client ID of the App User you created earlier>
    c. Client Secret: <Client secret of the App User certificate you created earlier>
    d. Tenant: <TenantID you saved earlier>

    IMPORTANT: The Service Principal connection must use the same Client ID as the **SPN Client Id** you set on the delegated deployment stage. If it uses a different app registration, the approval flow fails at Perform an unbound action with the error “Delegated deployments of type ‘ServicePrincipal’ may only be approved or rejected by the Service Principal configured in the deployment stage.” and the deployment stays Awaiting approval.



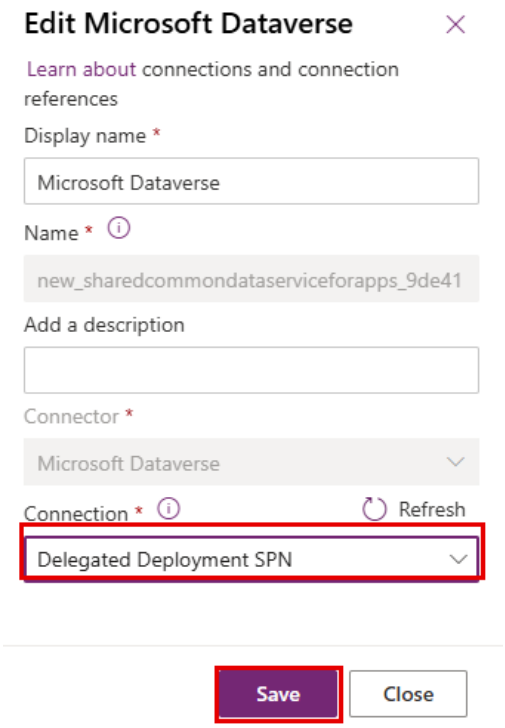
18. Do the same for the *If no* Dataverse action
19. Click **Publish**. Publishing saves the flow and turns it on.

Update Flow after solution import

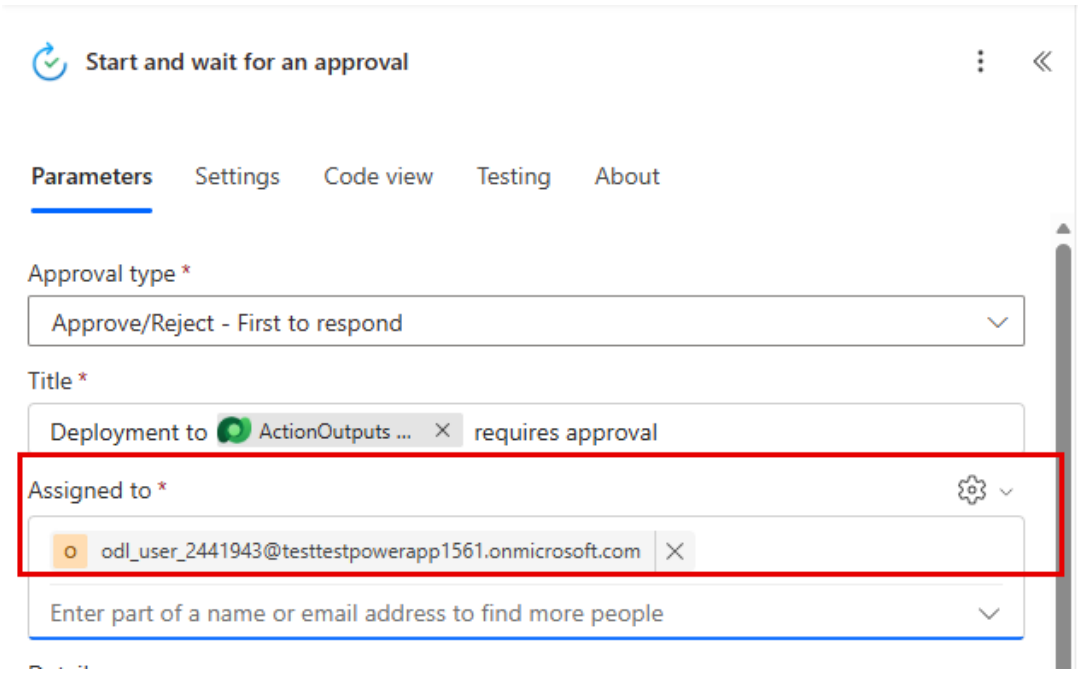
This section is only needed if you used the solution import method to create your flow. If you created the flow yourself, skip this section and continue on to the "Update pipeline to run as delegated" section.

1. After importing your solution, check the following items. If you need help with the steps, refer to the manual flow creation steps above.
    a. Ensure the **Trigger Condition** of your flow has the correct Pipeline name. The sample pipeline is configured for "Custom Host Pipeline" Update only if you used a different name for your pipeline. Here is a sample trigger condition: `@equals(triggerOutputs()?['body/OutputParameters/DeploymentPipelineName'], 'Custom Host Pipeline')`
    b. **Point the Dataverse connection reference to your SPN connection.** The trigger and both Perform an unbound action steps share one connection reference, so update it in the solution instead of changing each action: open the **Lab 3 Approval**

Delegation Flow solution > **Objects** > **Connection references** > **Microsoft Dataverse**. In the **Connection** dropdown, select your Service Principal connection, click **Save**, then click **Save changes**. If you do not have an SPN connection yet, create one first with the Client ID, Client Secret and Tenant ID from your app registration (see the connection steps above).



c. **Update the approver.** Open the flow, select the **Start and wait for an approval** step, remove the existing user in **Assigned to**, and add your own lab user. The imported flow is assigned to the original author's account.



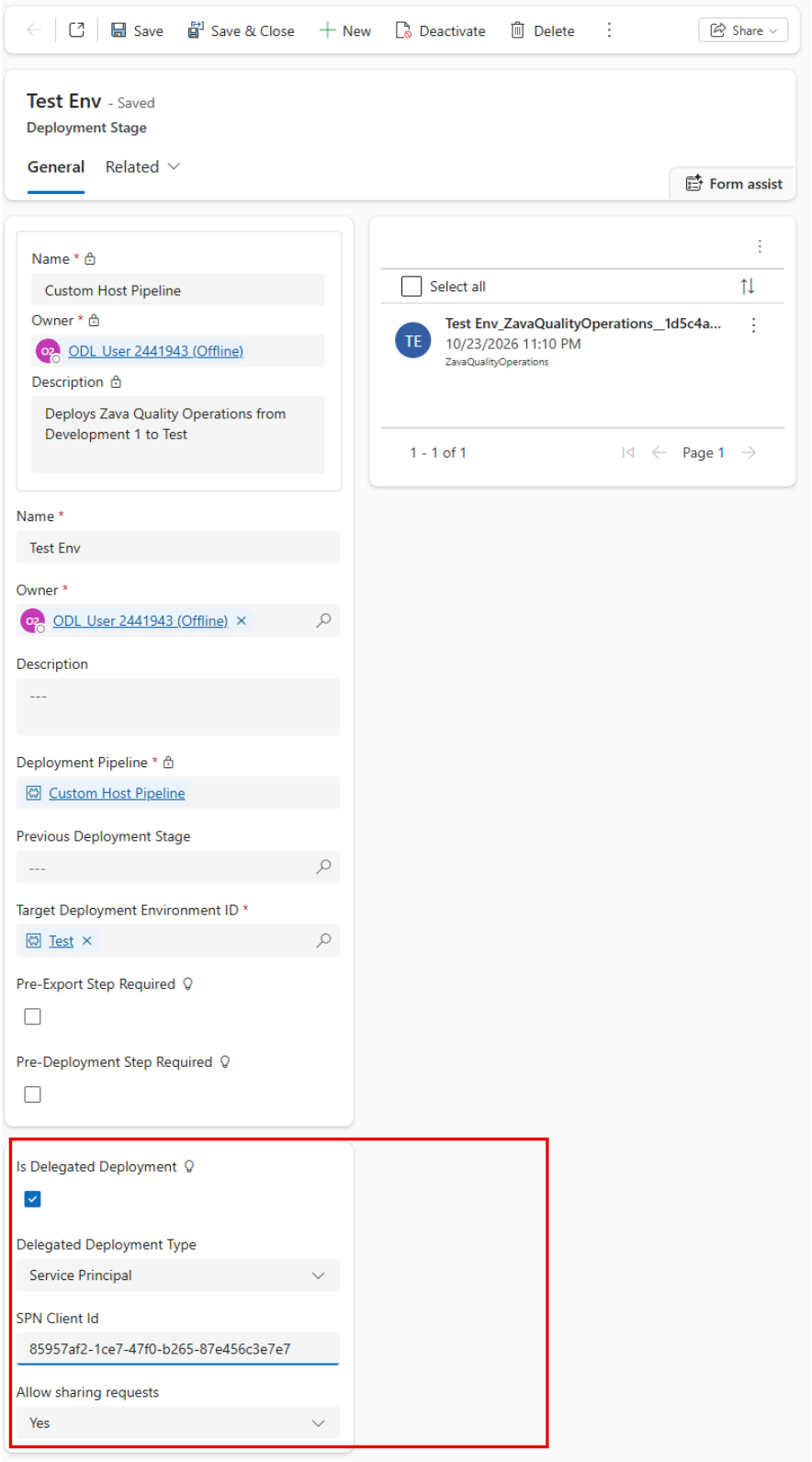
d. **Publish the flow and make sure it is on.** Click **Publish**, then go back to the flow details page and confirm the status is **On**. If it is Off, click **Turn on**.

IMPORTANT: The SPN connection must use the same Client ID as the **SPN Client Id** on the delegated deployment stage. Otherwise, the approval flow fails with "Delegated deployments of type 'ServicePrincipal' may only be approved or rejected by the Service Principal configured in the deployment stage." and the deployment stays Awaiting approval.

Update pipeline to run as delegated

The flow we just created in the previous section will not trigger until we enable the **Is Delegated Deployment** gated extension in the pipeline configuration. Now, let's configure our pipeline to initiate the flow.

20. Return to **Pipelines Host**, and launch the **Deployment Pipeline Configuration** app
21. Click on the **Pipelines** tab
22. Open the pipeline you created earlier (Custom Host Pipeline) by clicking on its name
23. In the **Deployment Stages (Deployment pipeline)** grid, open the **Test Env** stage.
24. On the deployment stage record form, check the box next to **Is Delegated Deployment**
    a. Select **Service Principal** for Delegated Deployment Type
    b. For SPN Client Id, provide the **client ID** of the app user you created earlier
    c. Allow sharing requests: **Yes**



25. Click **Save & Close**

Run SPN delegated pipeline

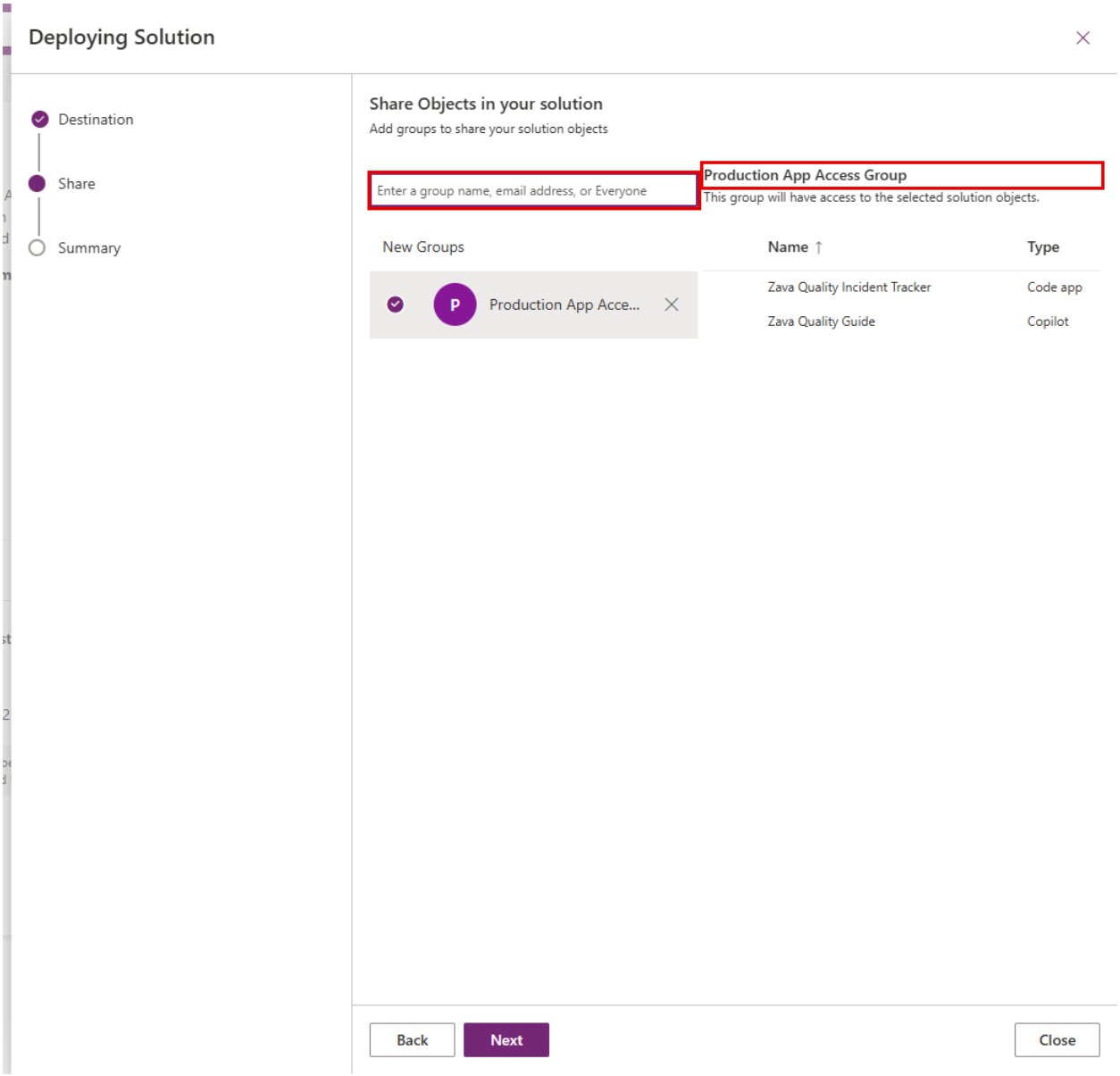
The **Share** step only appears when a deployment includes objects that are new to the target. Zava Quality Operations is already in **Test** from Task 1 and Task 2, so remove it first:

1. In Power Apps (https://make.powerapps.com/), select **Test** in the environment picker, then **Solutions** > **Managed**.
2. Select **Zava Quality Operations**, select **Delete**, and confirm. Wait until the solution no longer appears in the list.

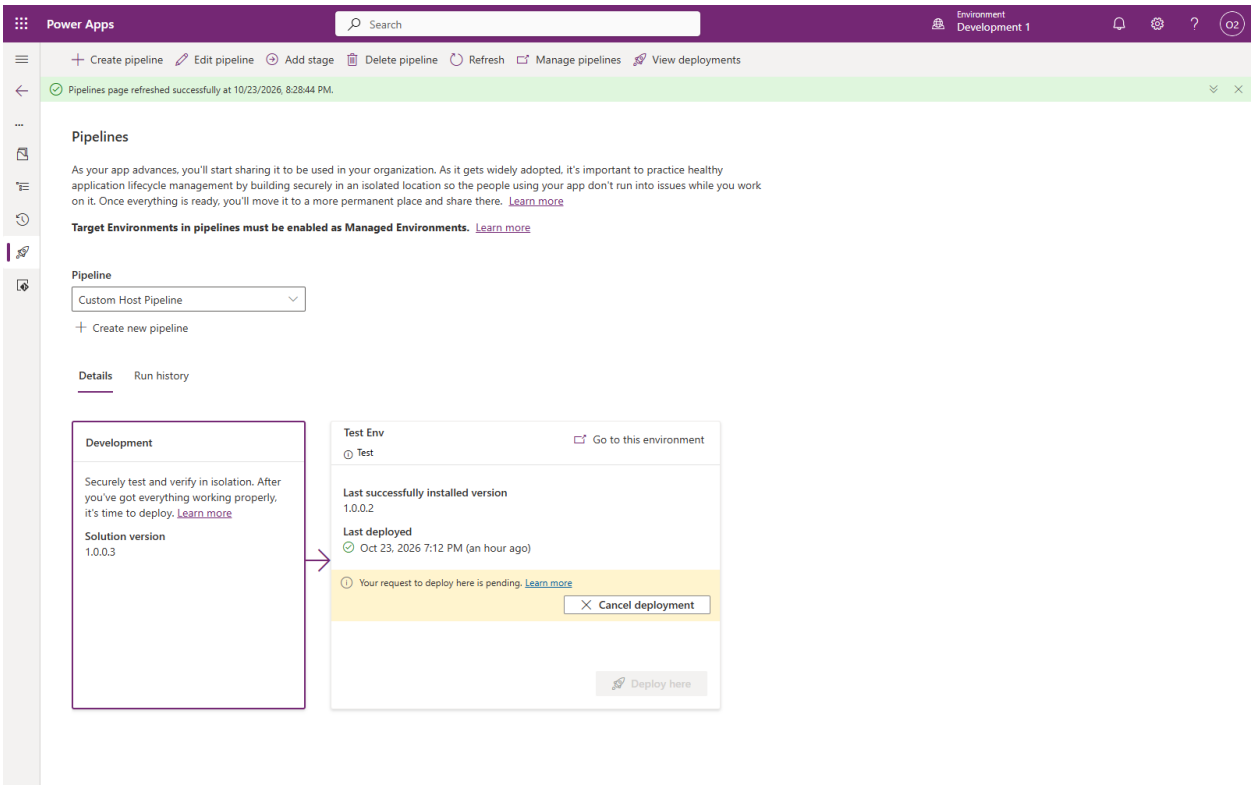
   IMPORTANT: Check that the environment picker shows **Test** before you delete. Don't delete anything in **Development 1**. Deleting the managed solution also removes its test records in **Test**.

Now test your delegated deployment. Go to **Development 1**, open **Zava Quality Operations** > **Pipelines**, select the custom host pipeline, and click **Deploy here** on the **Test Env** stage. Keep **Now** and click **Next**. You will see an additional step called **Share** which we didn't see when deploying using the Platform Host pipeline. This step will allow us to share objects within the Target environment directly from your Pipeline deployment. While the Zava Quality Operations solution doesn't have any connection references or environment variables, if the solution contains either, you'd see additional steps called "Connections" and "environment variables" respectively.

After the **Select deployment target** step, we see the Share step:

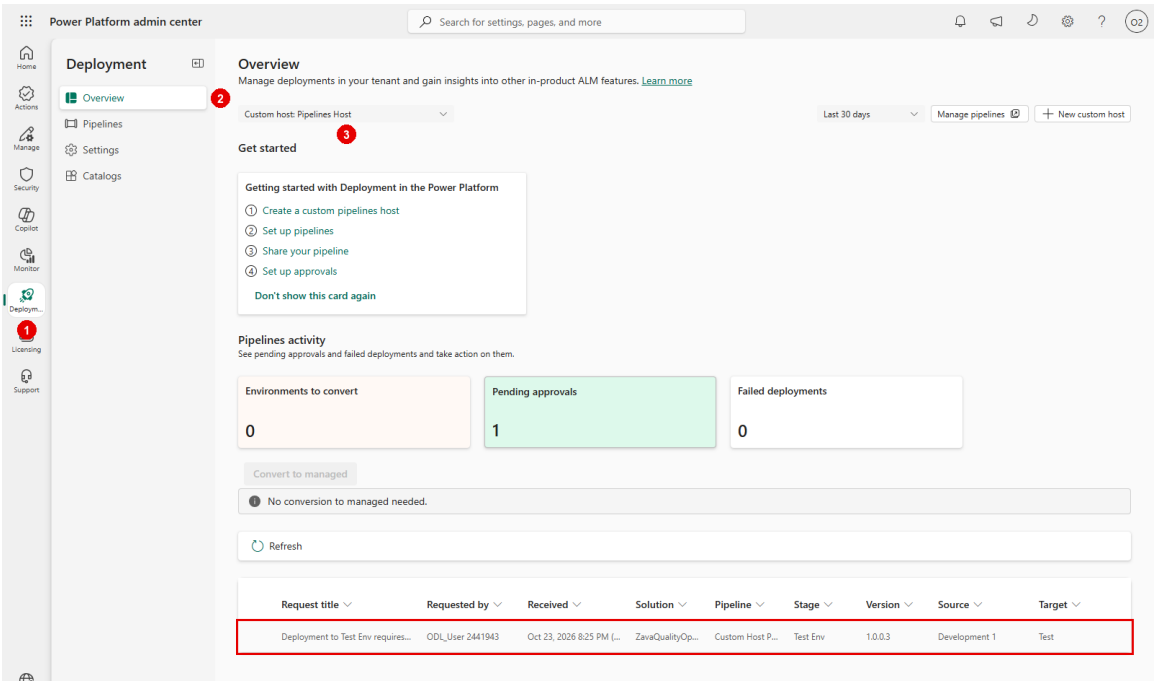


1. Search for **Production App Access** in the enter a group name box
2. Select **Production App Access Group**. The group will have access to the selected solution objects: **Zava Quality Incident Tracker** (Code app) and **Zava Quality Guide** (Copilot). There is no security role to select.
3. Click **Next**, then click **Deploy** on the next page. The Deployment stage will look like this as the pipeline waits for the approval process to complete:

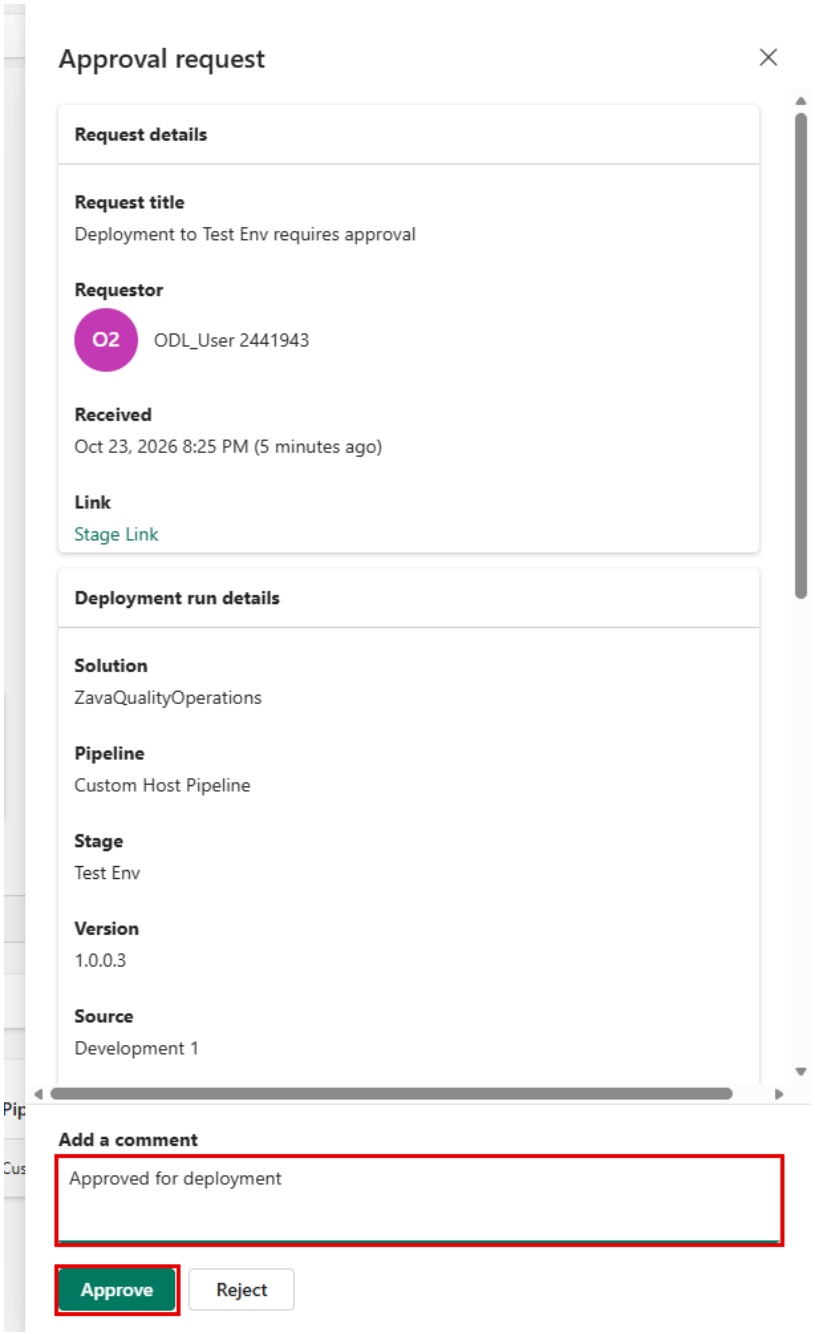


For the purpose of this lab, we will use the **Deployment** page in the Power Platform admin center (PPAC) to grant the approval. Normally, you have the option to pick from Teams, Outlook, or Power Automate portal to review the approval request.

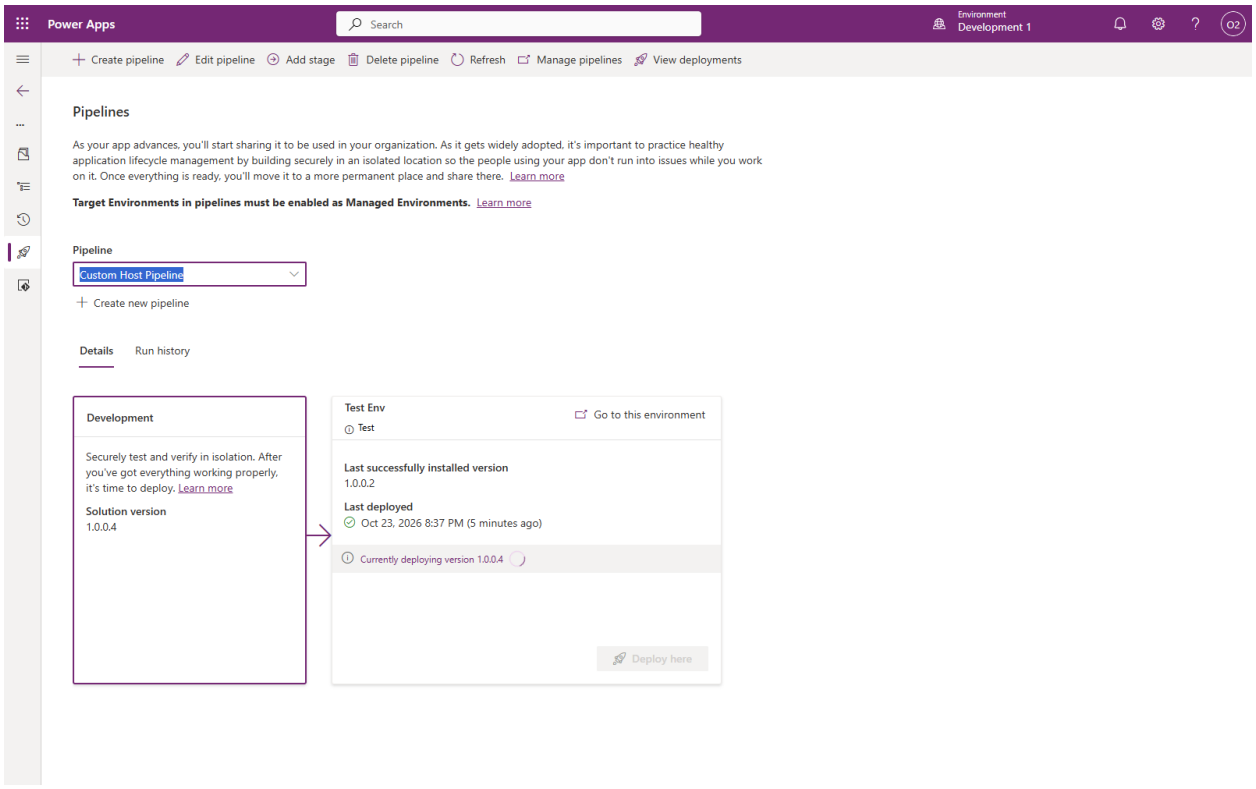
4. Navigate to https://admin.powerplatform.microsoft.com/ and click **Deployment** on the left navigation blade
5. On the **Overview** tab, select your custom host in the host dropdown (for example, **Custom host: Pipelines Host**)
6. Under Pipelines activity, the **Pending approvals** tile shows your pending approval, and the request is listed in the table below it



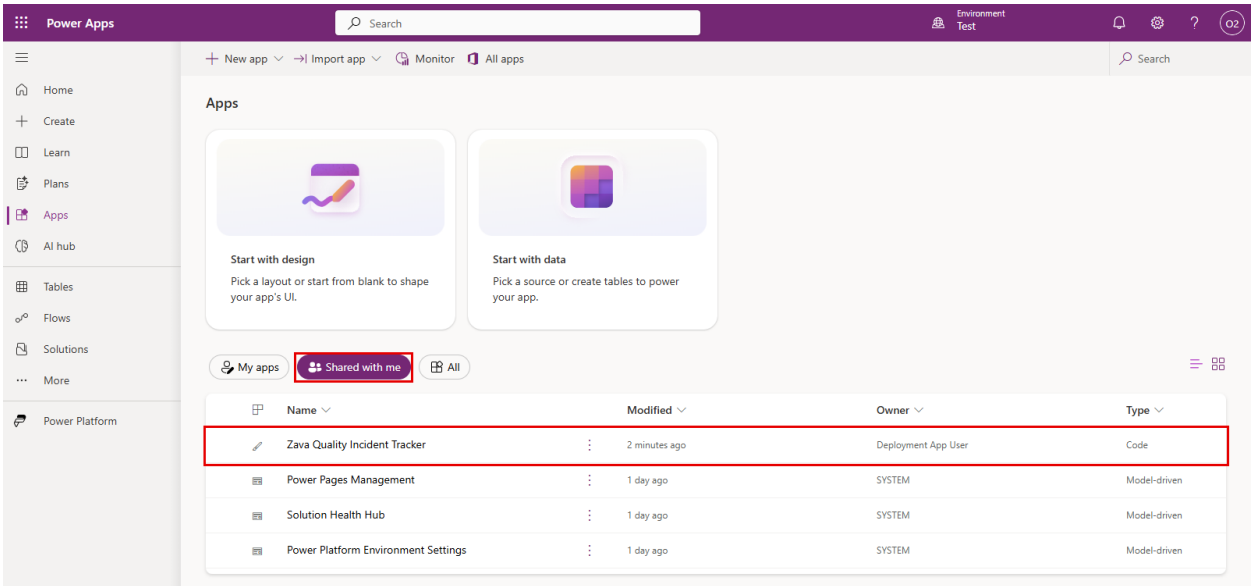
7. Click on the request to review it. The **Approval request** pane will appear on the right hand side of the screen. Optionally add a comment, then choose **Approve** here:



8. Check back on your pipeline after receiving the approval. Your **Test Env** stage will update with “Currently deploying version #.#.#.#”



9. Once complete, click on the **Go to this environment** hyperlink on the deployment stage
10. Click on **Apps** in the left-hand navigation
11. Select **Shared with me**
12. Verify that **Zava Quality Incident Tracker** is listed. Its owner is **Deployment App User** because the deployment ran as the service principal.
13. Because the app was shared with the **Production App Access Group** during deployment, members of the group (including you) can open it from here.



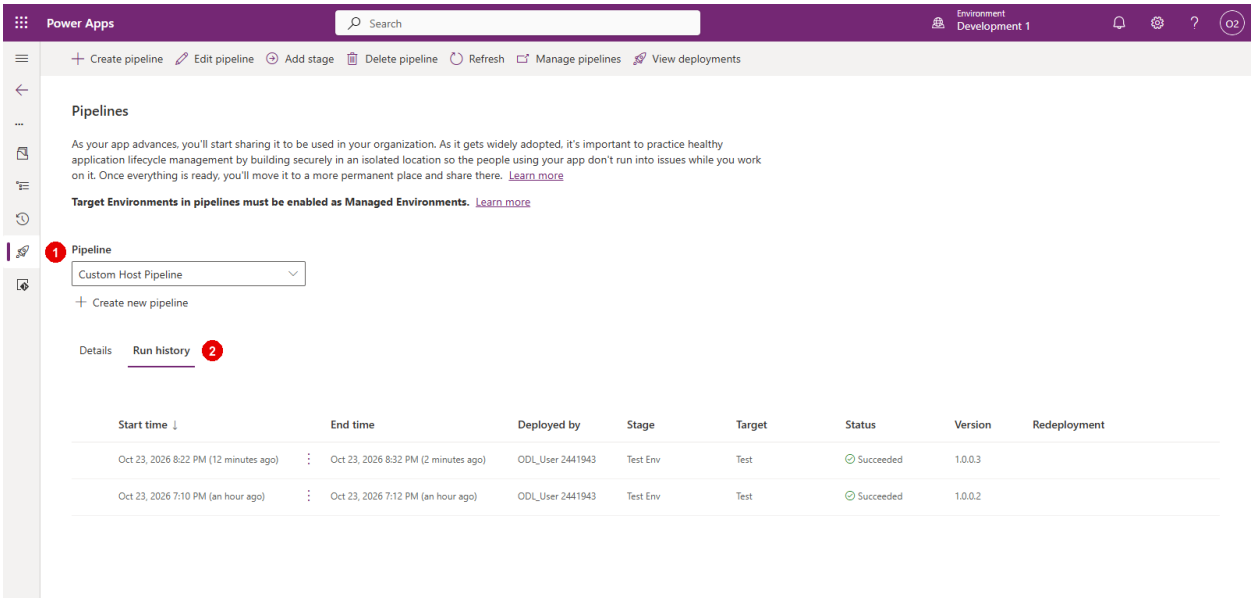
Redeploy a prior version

- **Prerequisite:** If you didn't complete the Extend Pipelines task, submit another deployment now. Run History should have at least two deployed versions.

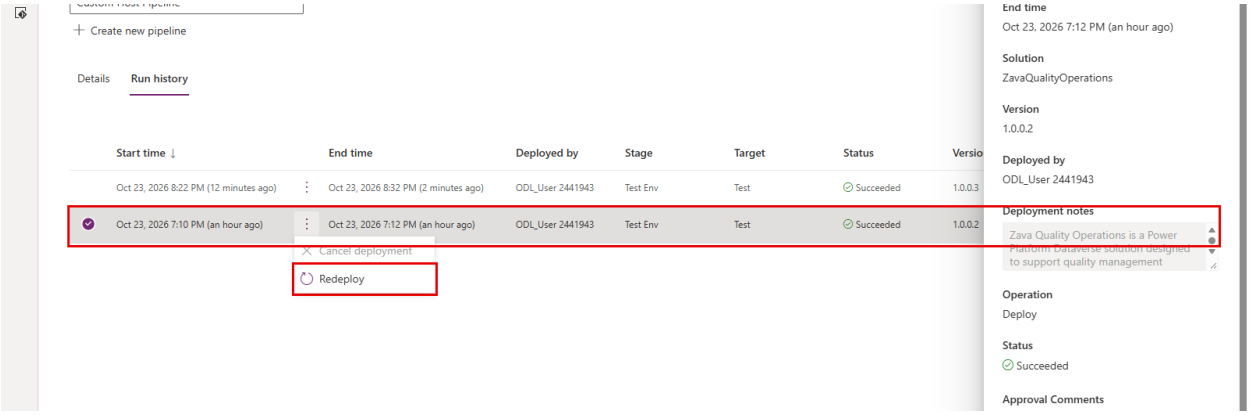
While delegated deployments and approvals reduce the chances of breaking applications in production, mistakes can and do happen.

Let's pretend the last deployment contained a critical breaking change and it needs to be rolled back. In this task, you'll re-deploy the last known good version of your solution.

1. Open the maker portal (https://make.powerapps.com/) and make sure you are in **Development 1**
2. Open **Solutions** from the left hand navigation blade
3. Open the unmanaged solution (Zava Quality Operations) that you deployed using the pipeline earlier in this lab
4. Select **Pipelines** > click on the **Run History** tab



5. Select the lower version, then **Re-deploy** from the menu.



6. Select **Redeploy** in the confirmation popup



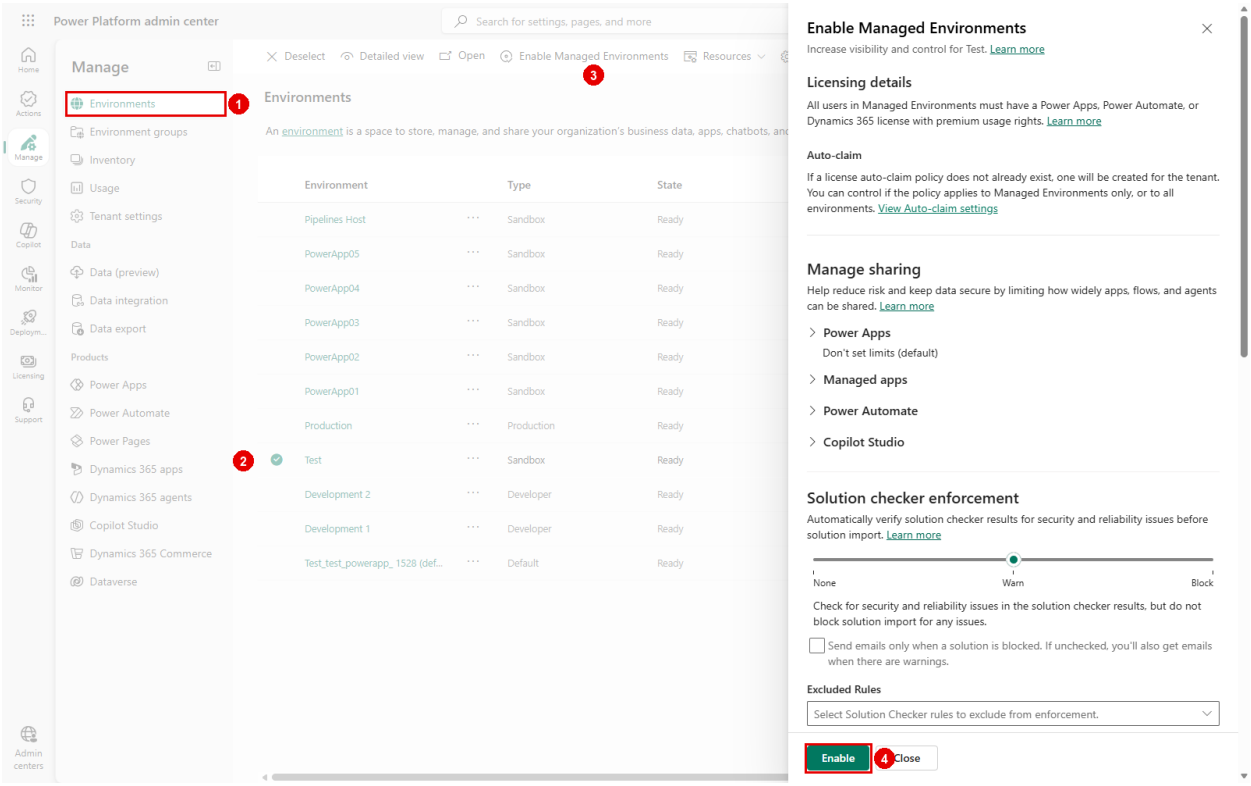
7. When the deployment panel opens, click Next until you get to the last screen, then **Deploy**. *See prior Task for detailed steps if needed.*
8. In a new tab, open the **Deployment** page in the Power Platform admin center again and **Approve** the deployment request
9. Deployment is now in-progress. Wait until it finishes. *You may need to refresh the pipelines page.*
10. Disaster averted! Now the prior version is deployed to **Test** and you have time to fix the (hypothetical) issue and deploy an updated version when ready.

Appendix:

Steps to enable Managed Environments

1. Navigate to Power Platform Admin Center (PPAC) at https://admin.powerplatform.microsoft.com/manage/environments

2. In the Manage tab on the left hand side, click on **Environments**
3. *Select* the environment where you want to enable Managed Environments
4. In the ribbon, click **Enable Managed Environments**
5. Click **Enable** in the fly out wizard



6. Repeat steps 3-5 for any other environments where you need to enable Managed Environments.